Announcements, Sep 8, 2025

- All recorded lectures and lecture notes on the USC Brightspace, see https://brightspace.usc.edu/
- We will have 3 guest lecturers this semester (see revised syllabus):
  - September 15, 2025: Dr. Hieu Nguyen of eBay on key-value stores, graph databases, and dataflow execution paradigms for AI workloads.
  - September 22, 2025: Dr. Haoyu Huang of DataBricks on separation of storage from processing and disaggregated DBMSs for AI agents.
  - November 3, 2025: Dr. Douglass Terry of LinkedIn on Paxos.
- CSCI 585 material can be used to win an award & provide real solutions!
  - Global Space-Based Challenge, $150K prize money. Deadline is Oct 26, 2025 (7 weeks from today). See https://www.fire.ca.gov/what-we-do/wildfire-technology/development-challenge for details. Form a group, get involved, send me (shahram@usc.edu) an email if you are interested in pursuing.
  - This lecture and the next two lectures are very relevant.

Gamma

by DeWitt et. al.

Shahram Ghandeharizadeh
CSCI 585

ACM Software Systems Award

Awarded to an institution or individual(s) recognized for developing a software system that has had a lasting influence, reflected in contributions to concepts, in commercial acceptance, or both.

Gamma is the recipient of the ACM Software System Award 2008.

Gamma continues to influence systems designed today.

Why is Gamma Relevant to Today's Systems

Gamma was designed for the likes of Facebook, Amazon, eBay, Azure, and other giants of the Internet before they existed.

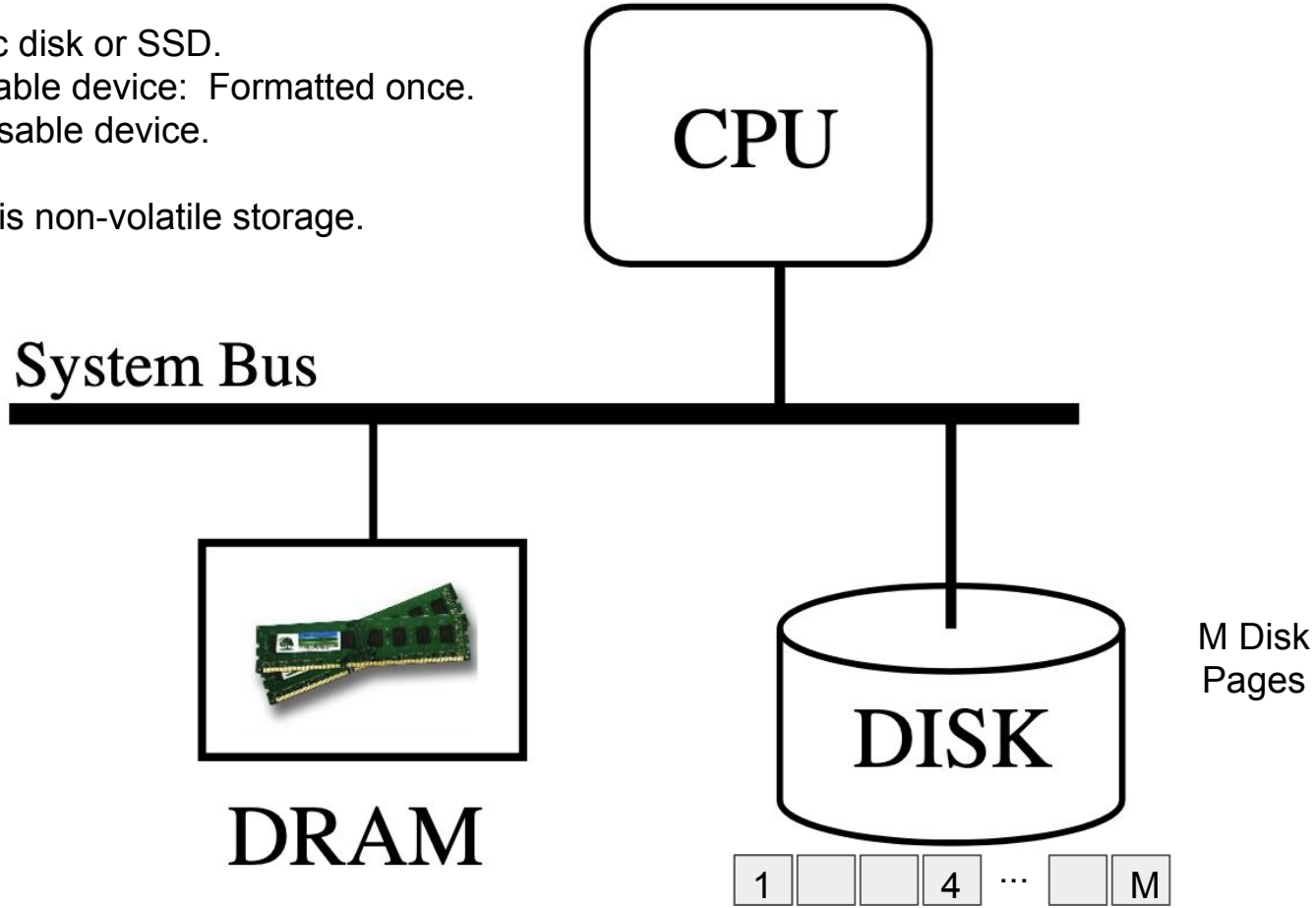
Gamma hides parallelism using SQL. The user who writes the SQL query is not aware that it is being executed in a parallel manner using many computers. Today's comparable systems include Amazon RedShift and Google BigQuery.

Node Hardware & Disk I/O

Hardware Organization

Disk may be a magnetic disk or SSD.
Disk is a block addressable device: Formatted once.
DRAM is a byte-addressable device.

DRAM is volatile. Disk is non-volatile storage.

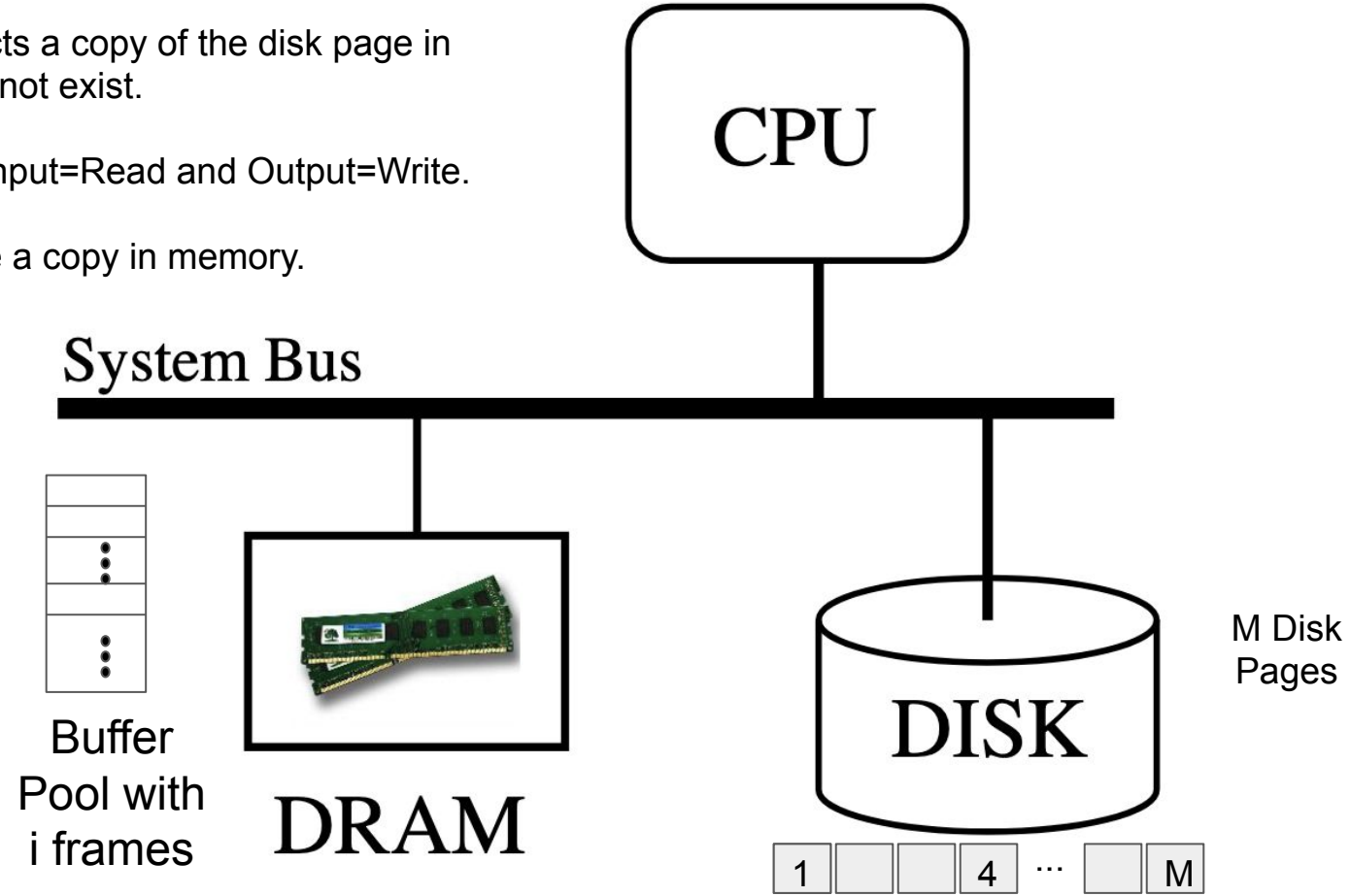


Disk Input/Output (I/O)

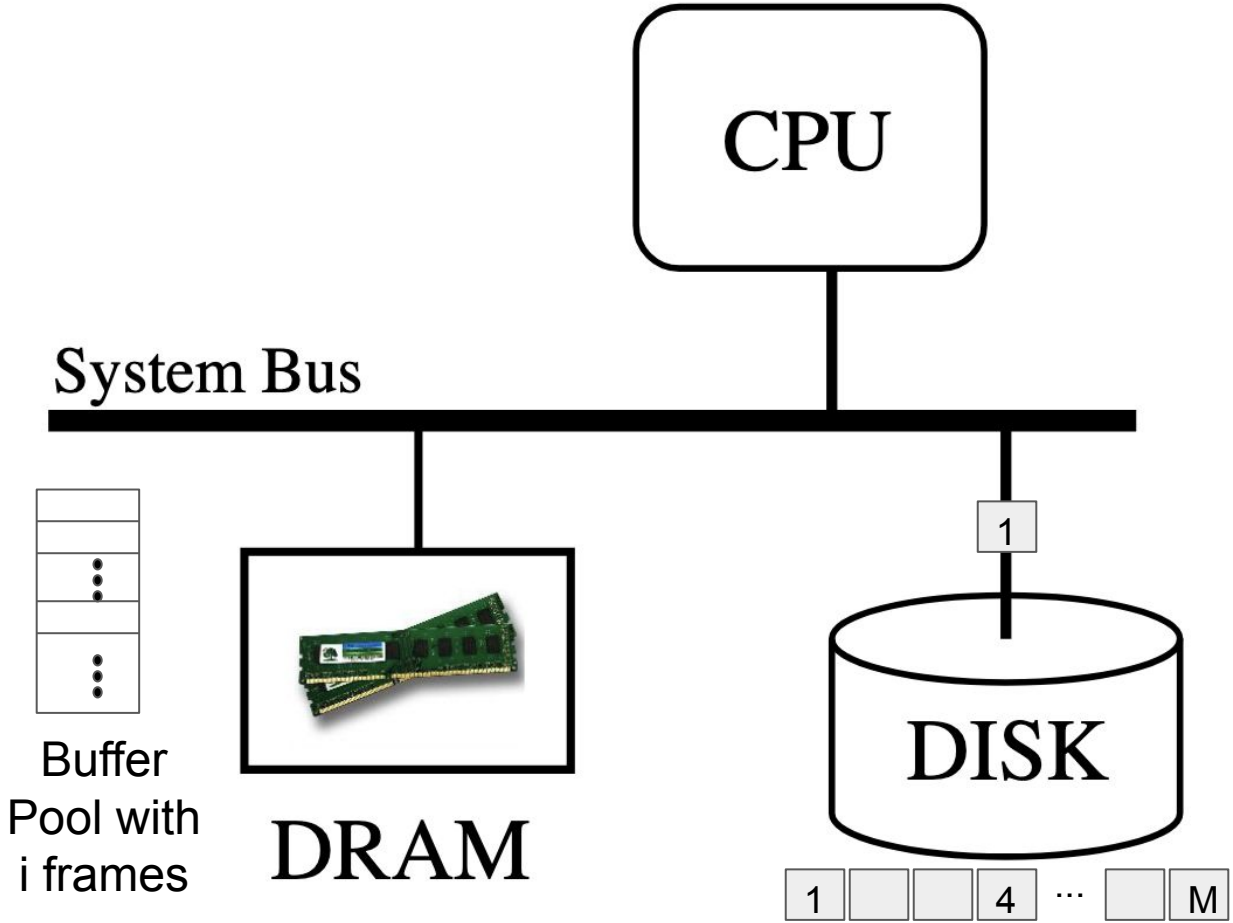
A Disk read constructs a copy of the disk page in memory if one does not exist.

Disk I/O stands for Input=Read and Output=Write.

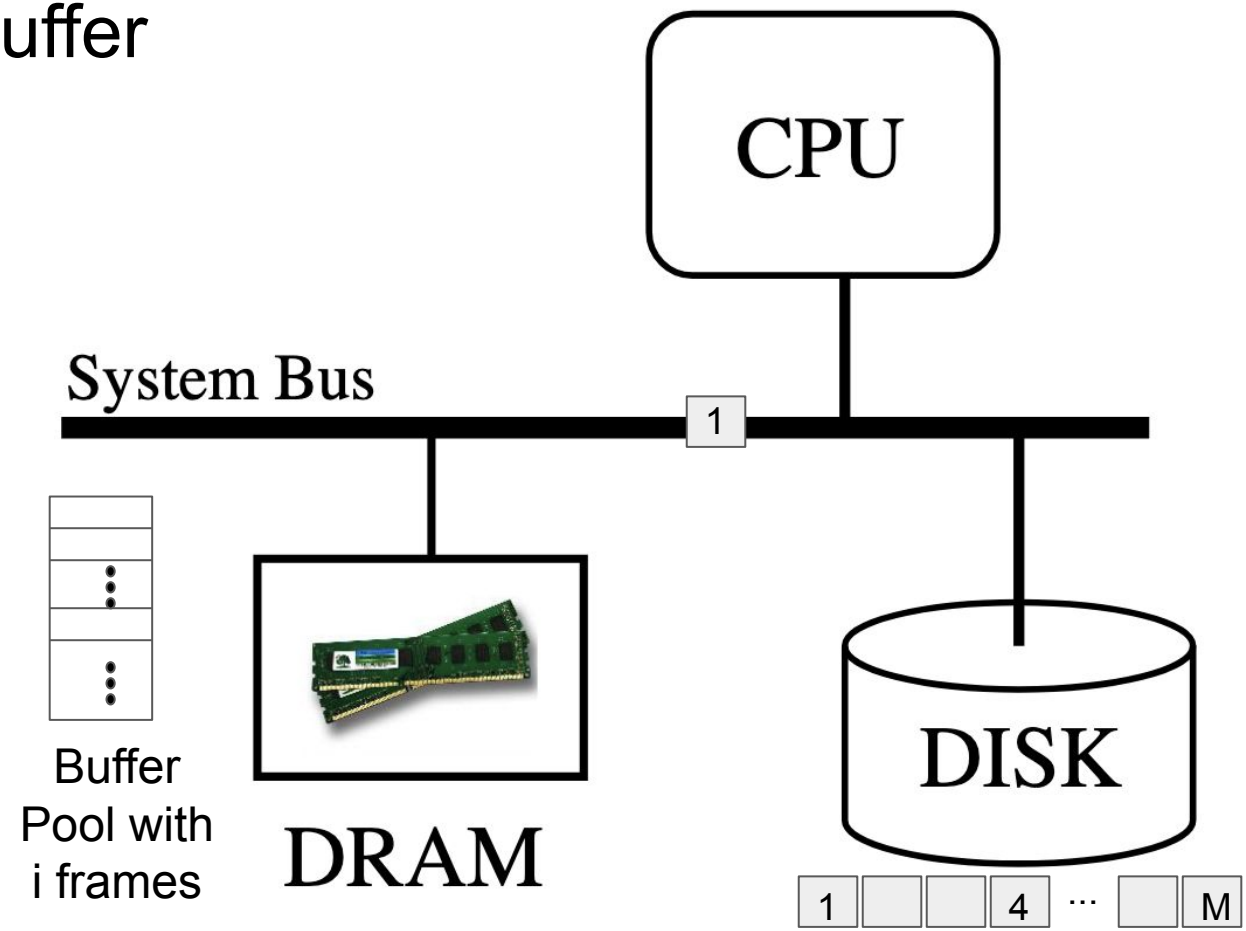
Disk write may leave a copy in memory.



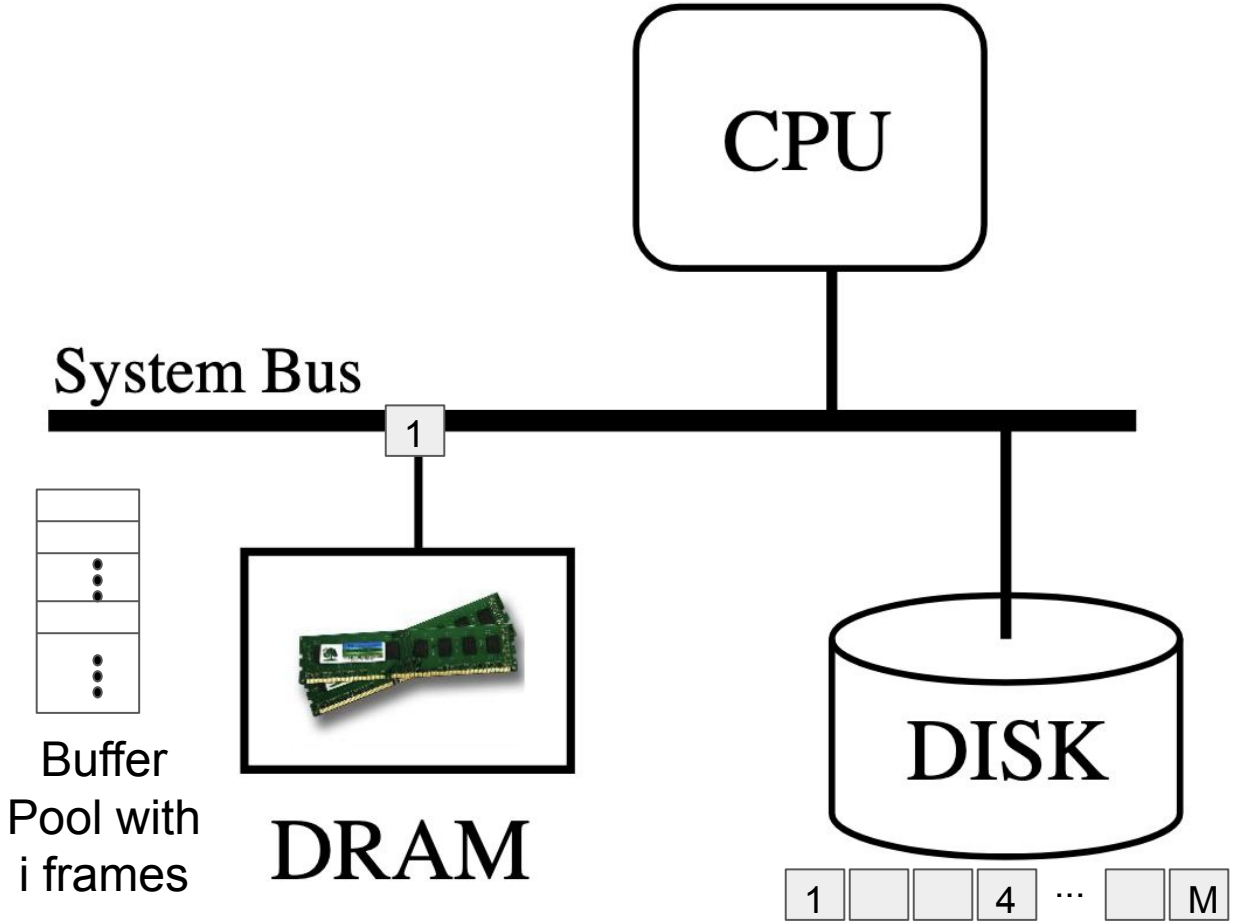
Disk Input/Output (I/O): Read Disk Page 1



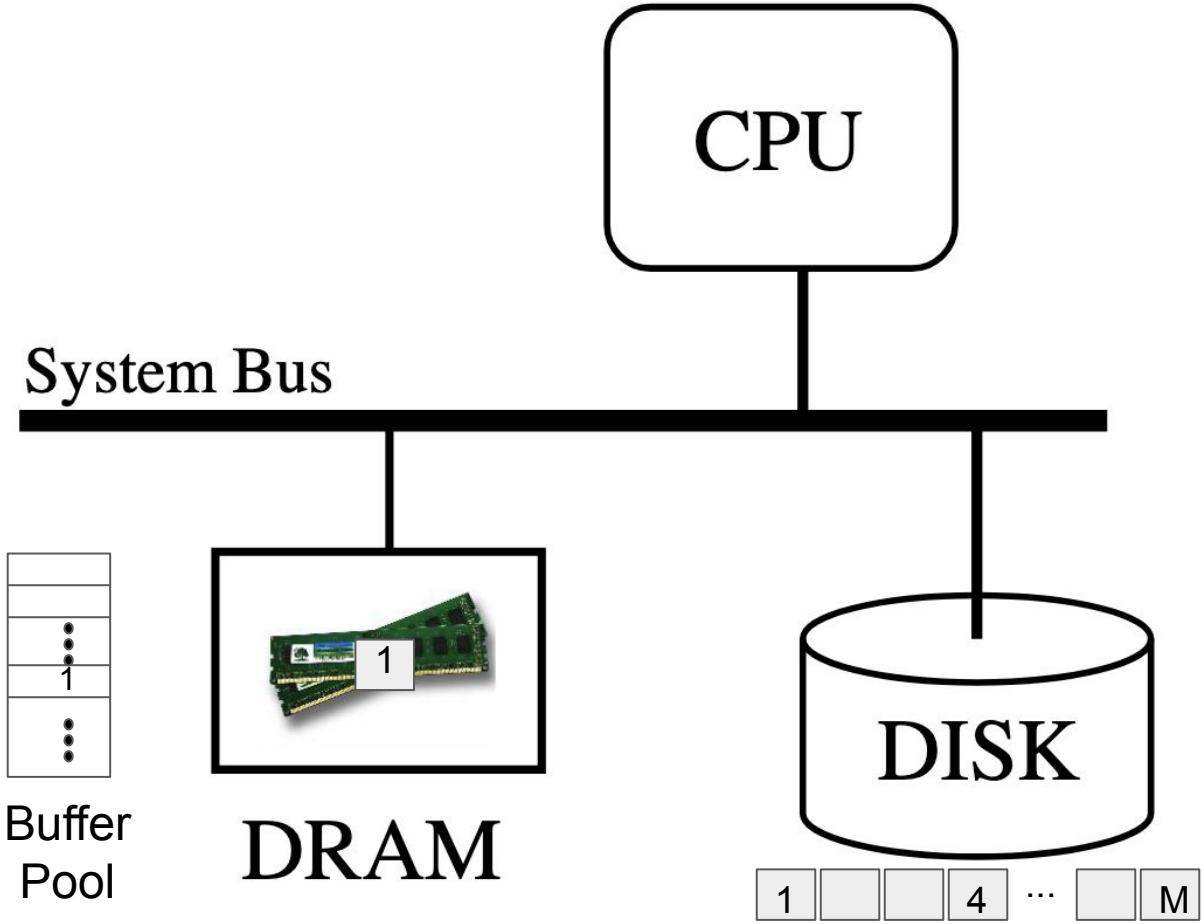
Disk Input/Output (I/O): Page 1 is Written into a Memory Buffer



Disk Input/Output (I/O)

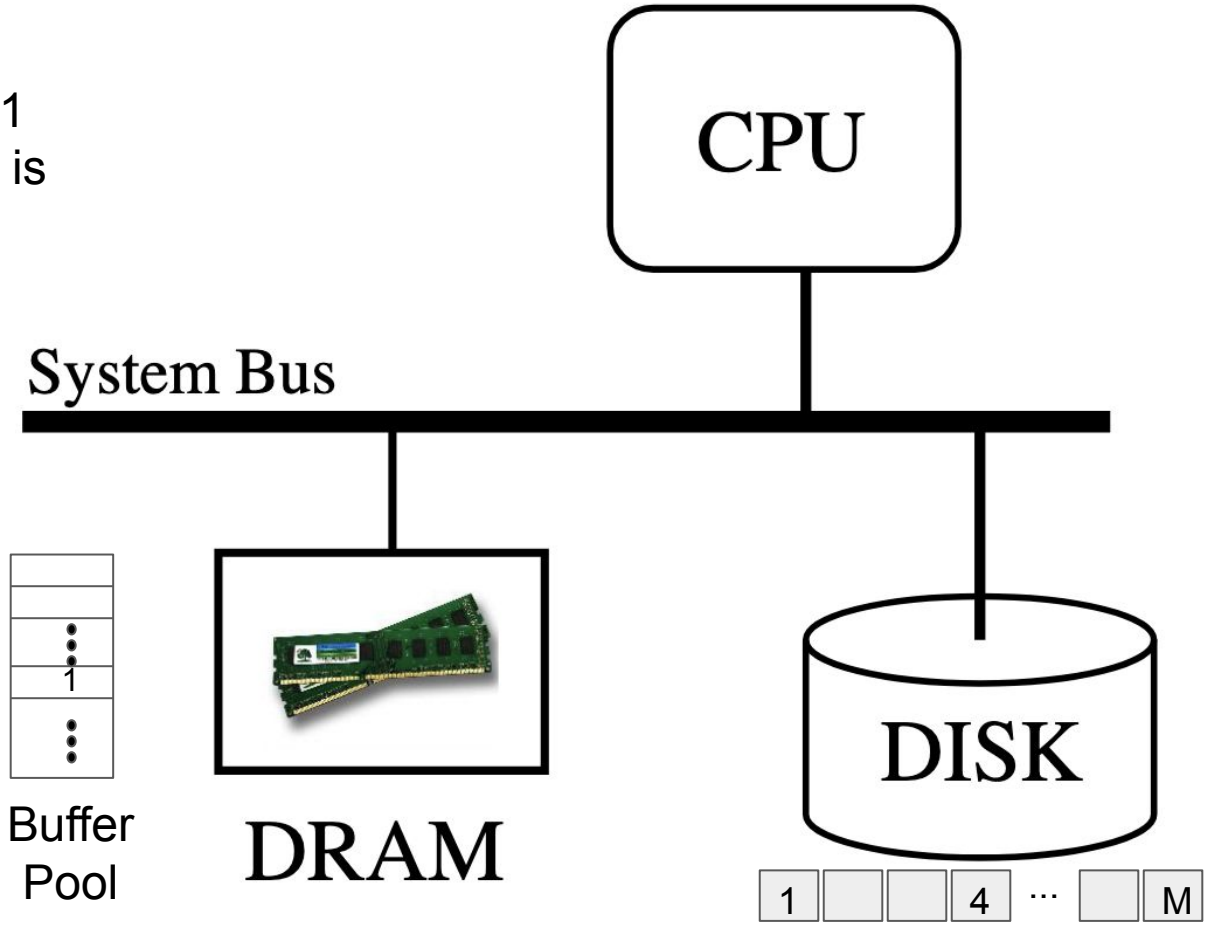


Buffer Pool Consists of Disk Page Frames

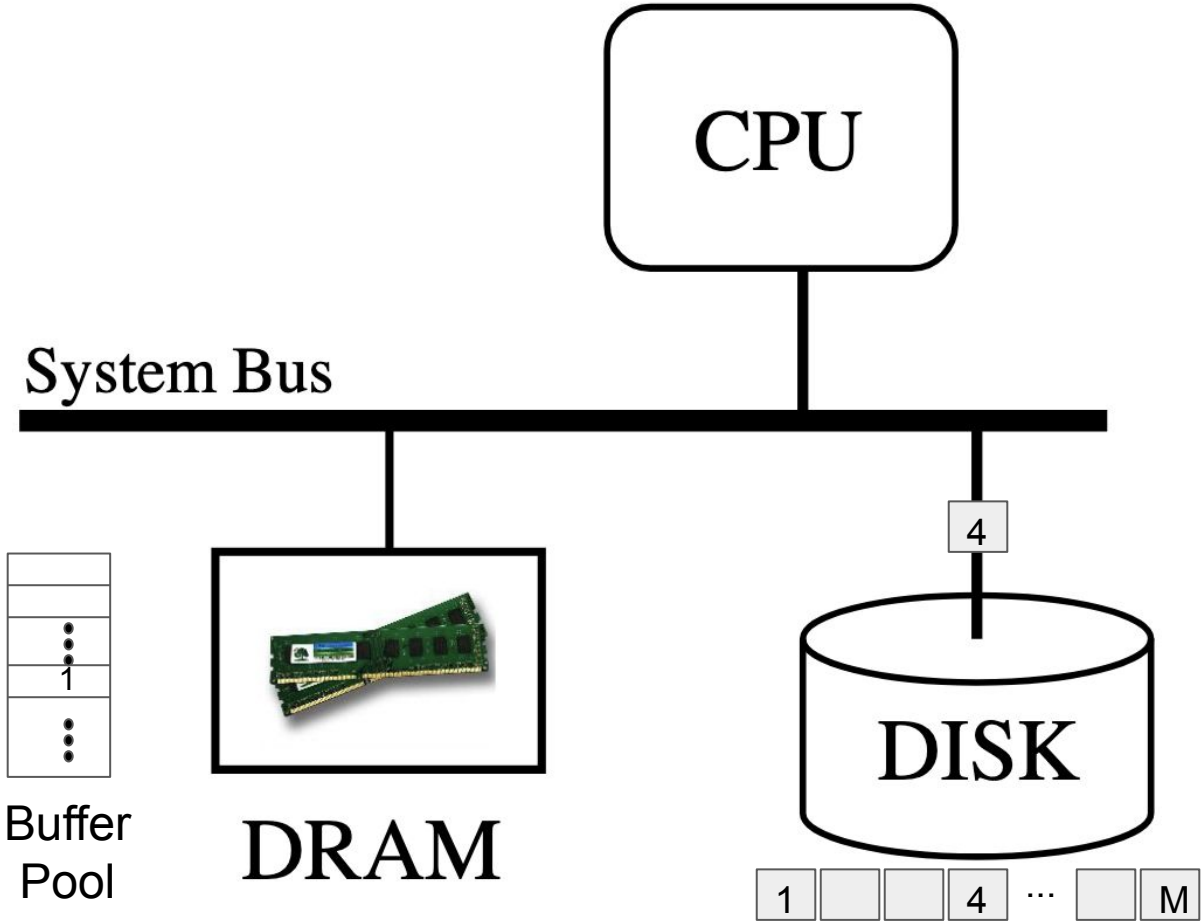


A Disk Page Occupies a Frame

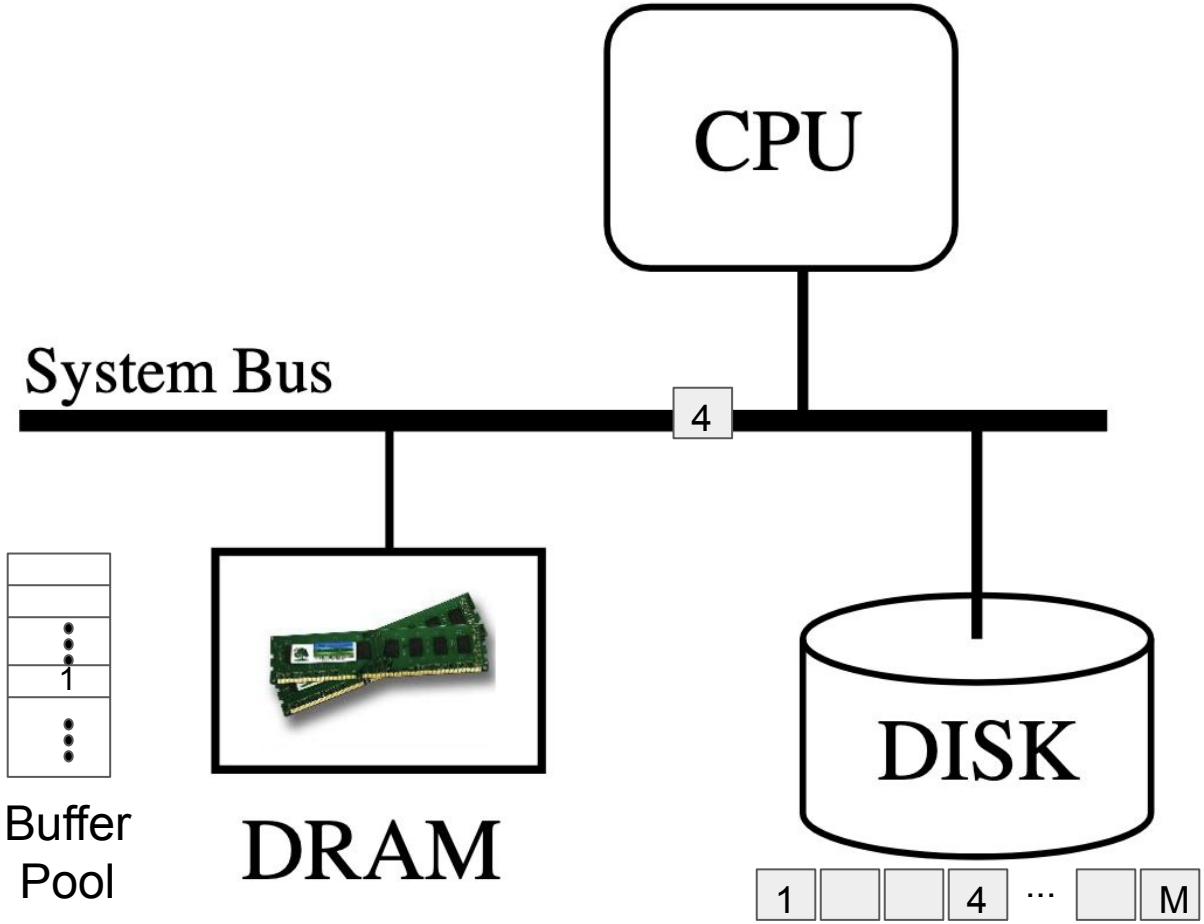
How to find Page 1 in memory once it is referenced?



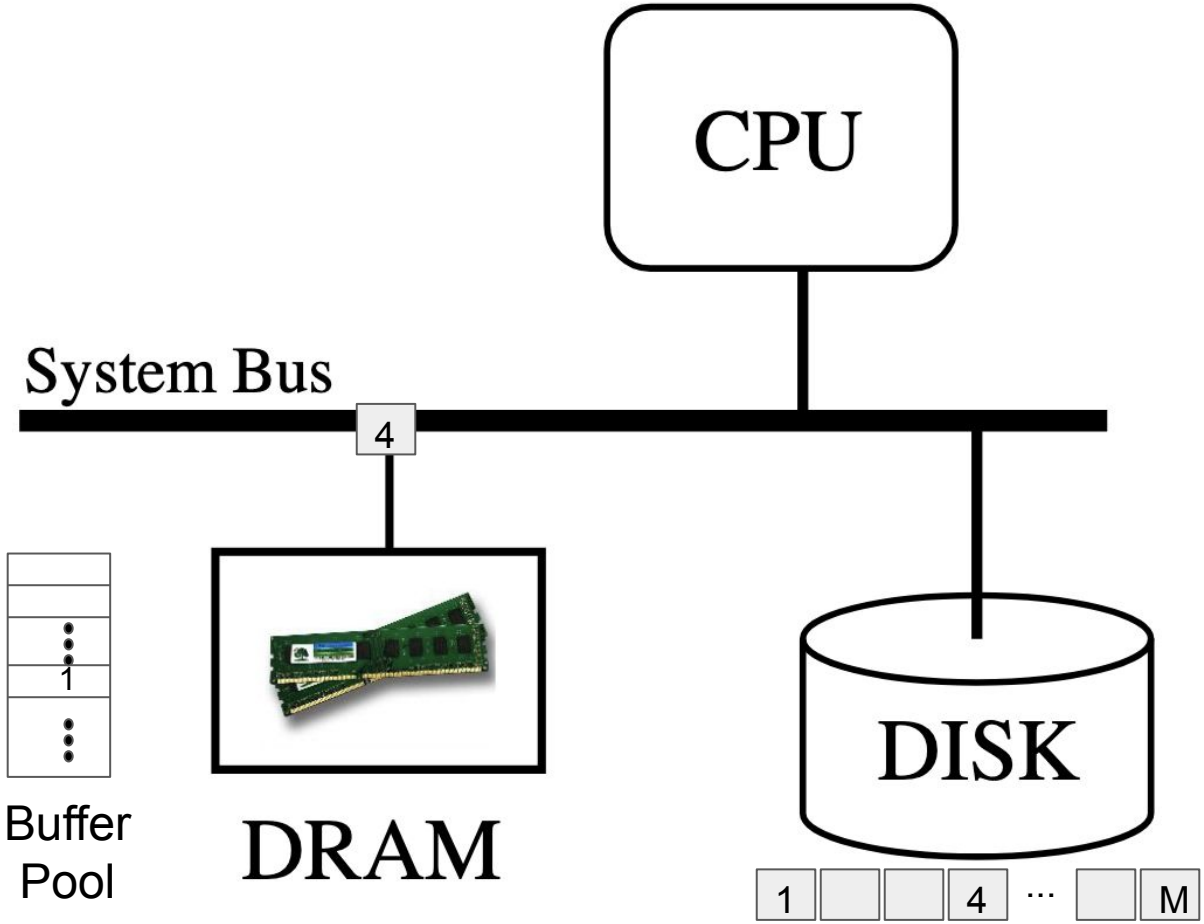
Disk Input/Output (I/O): Read Page 4



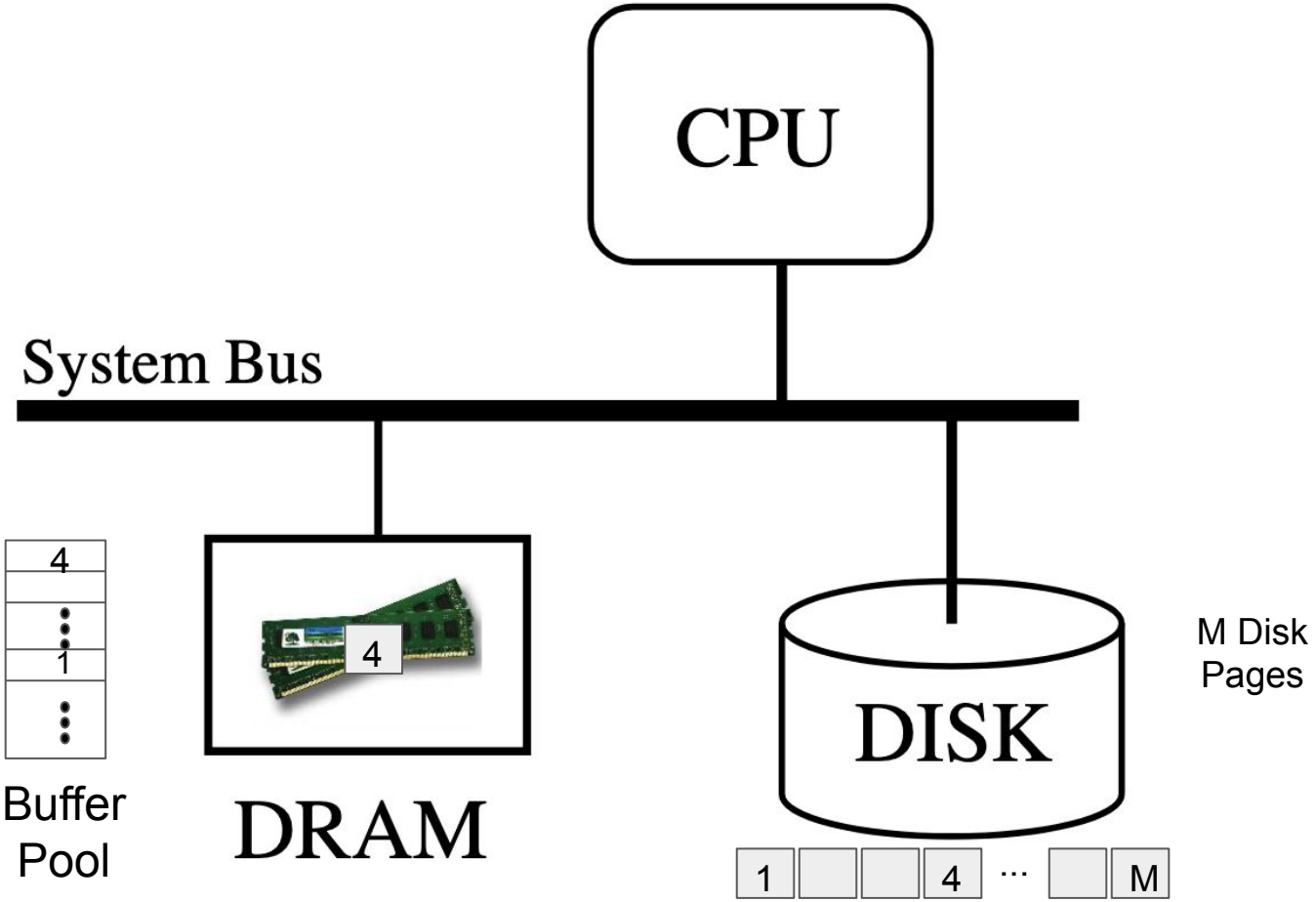
Disk Input/Output (I/O)



Disk Input/Output (I/O)



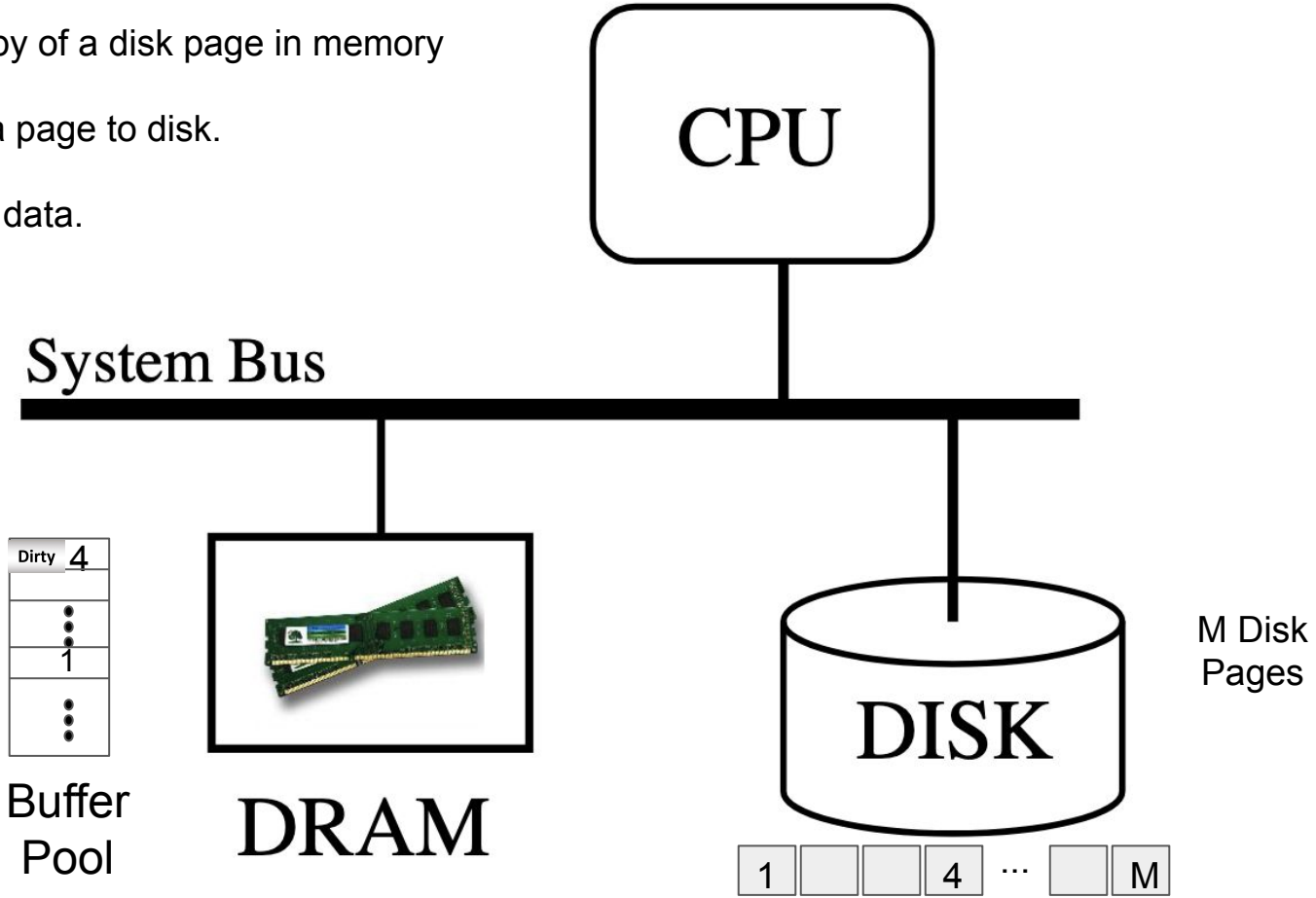
Each Disk Page Occupies a Buffer Pool Frame



Disk Input/Output (I/O)

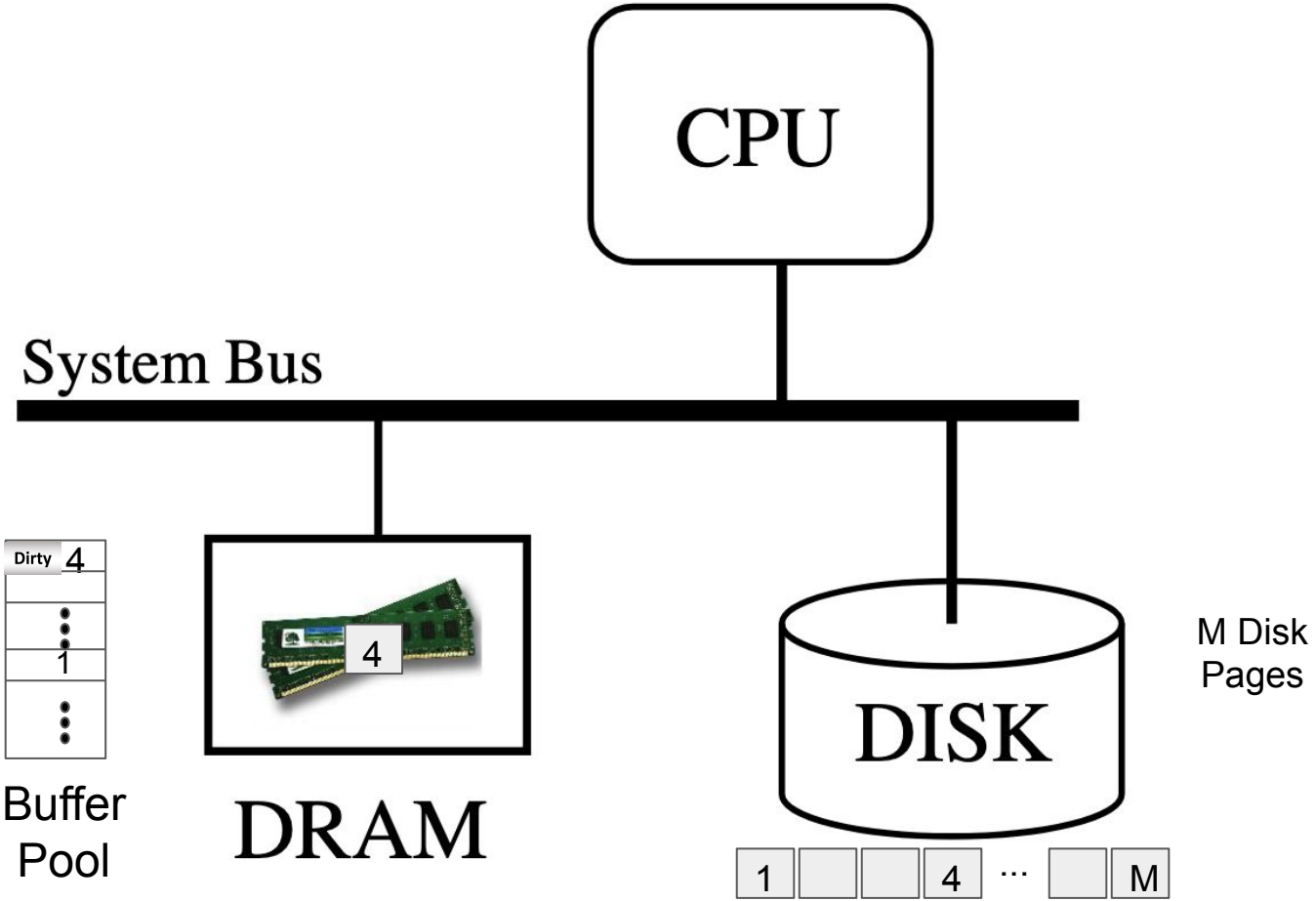
A write updates the copy of a disk page in memory and marks it dirty.
The DBMS may flush a page to disk.

Write Page 4 with new data.

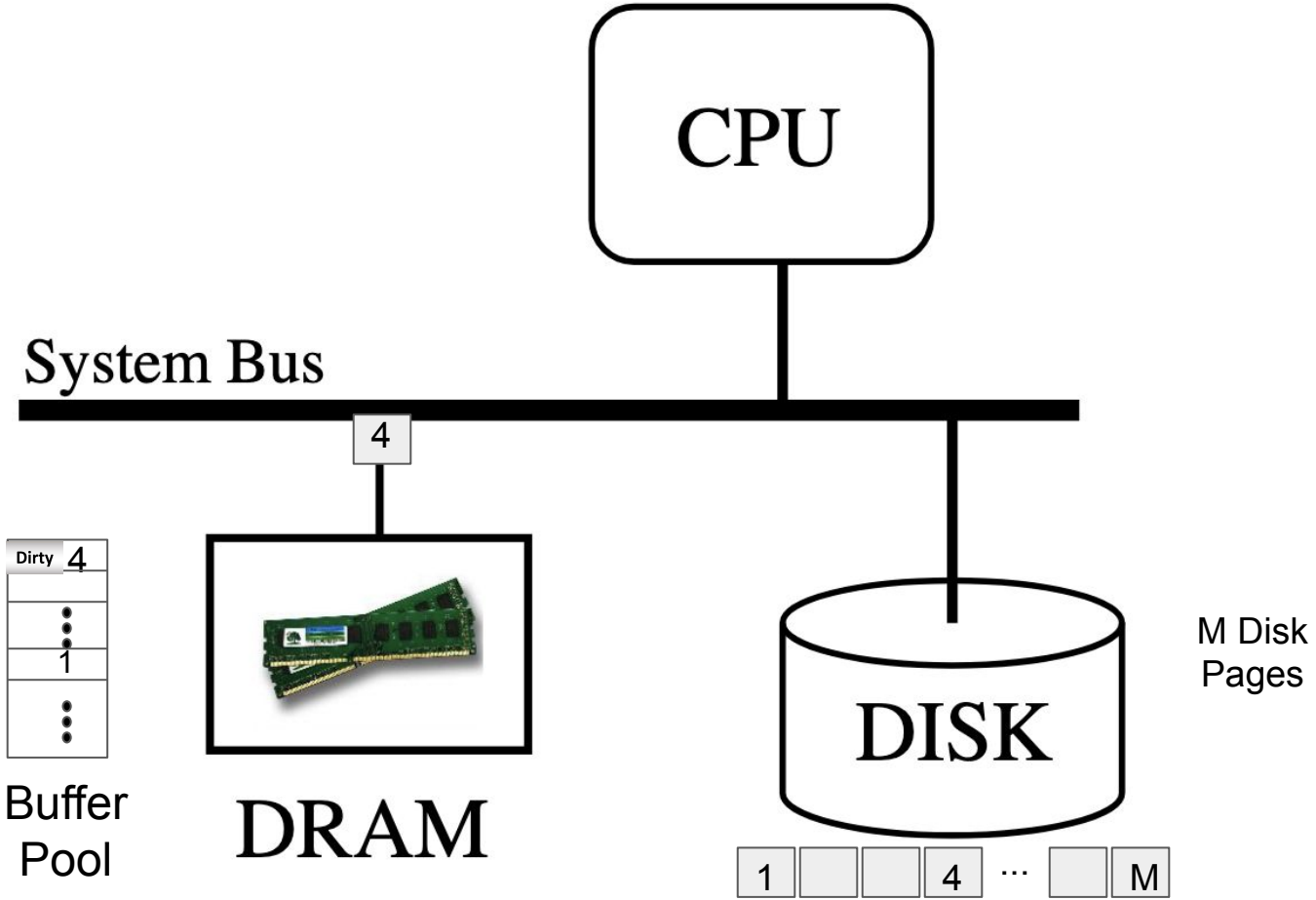


Disk Input/Output (I/O)

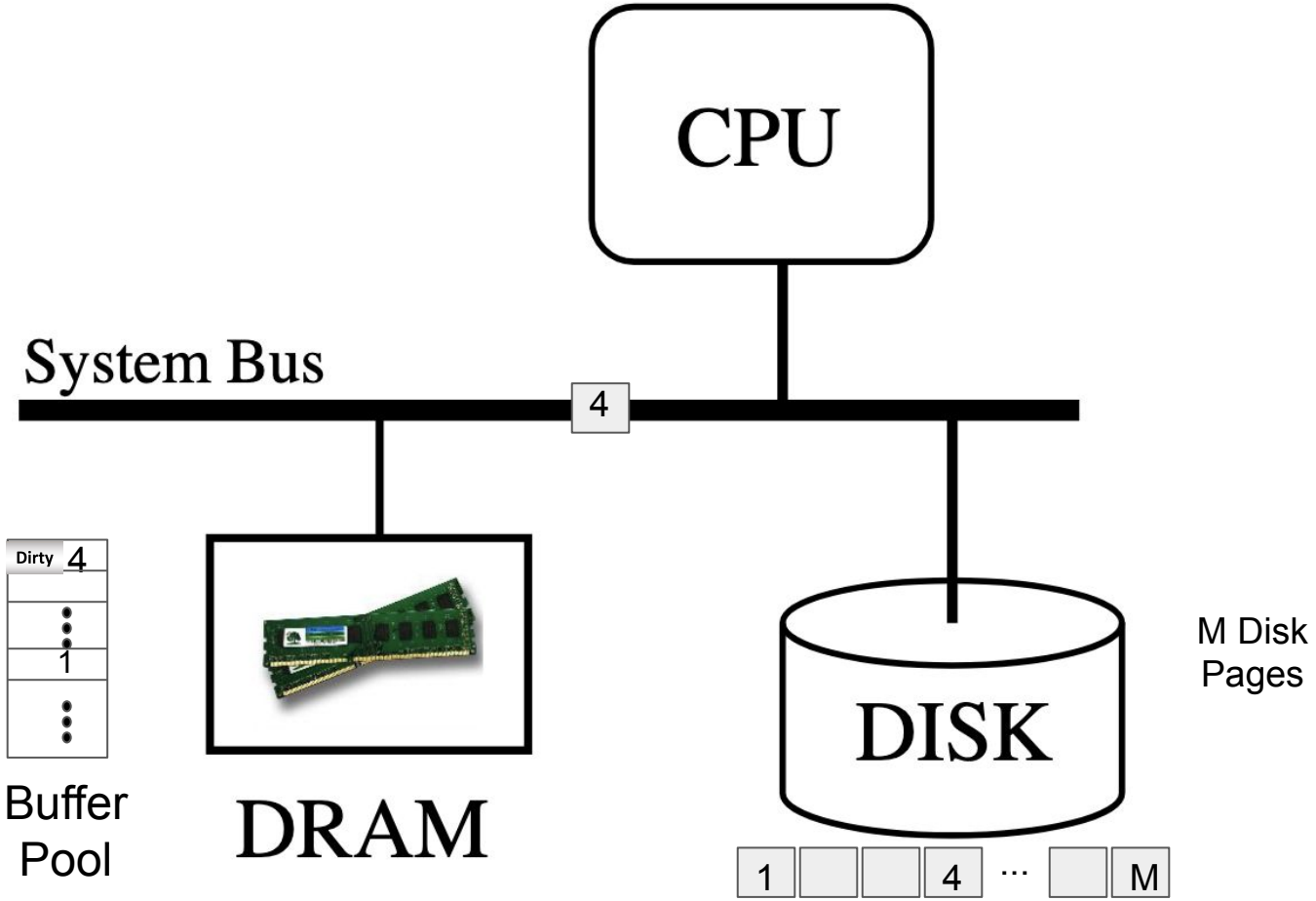
Flush Page 4 to Disk.



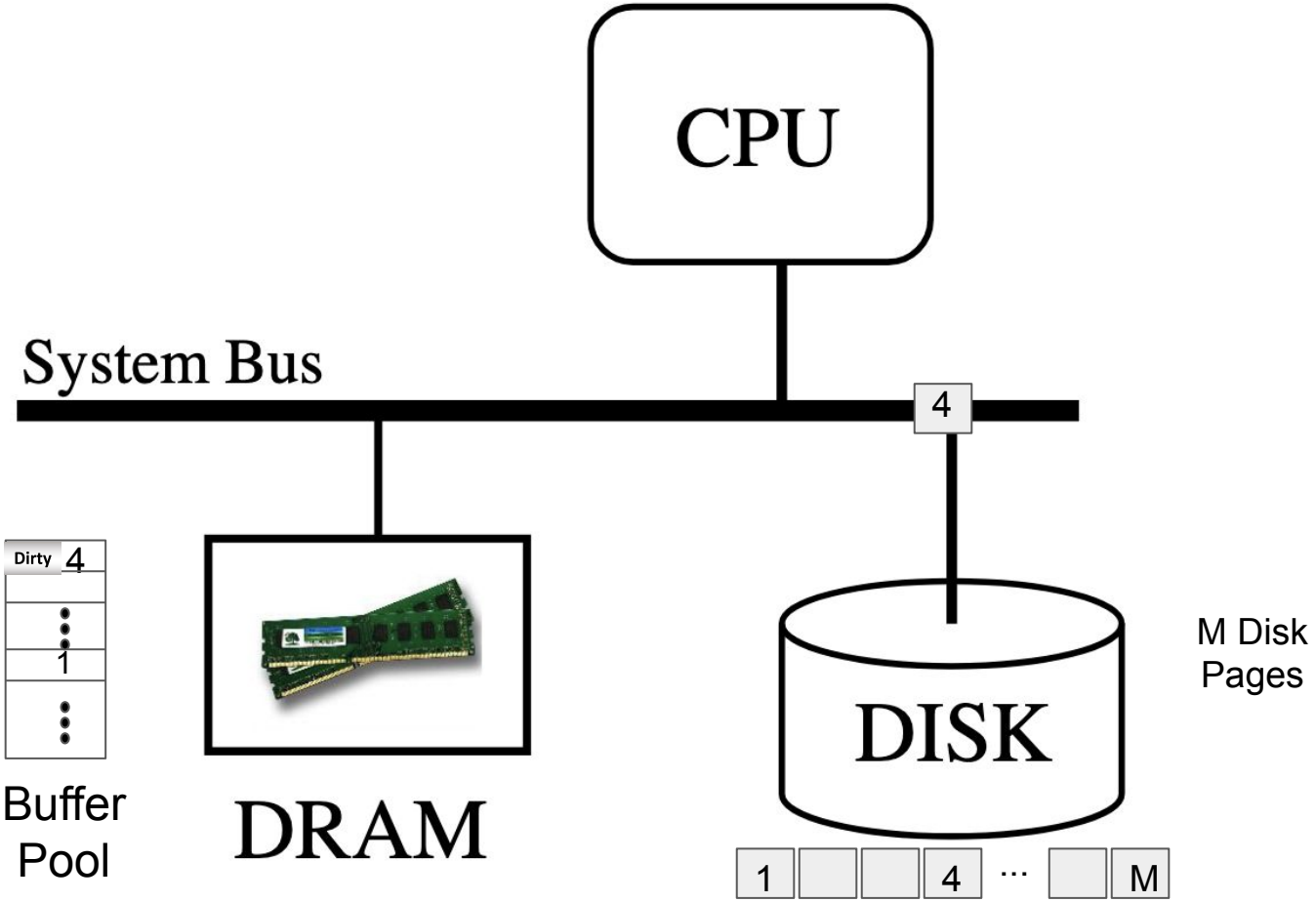
Disk Input/Output (I/O): Flush Page 4 to Disk



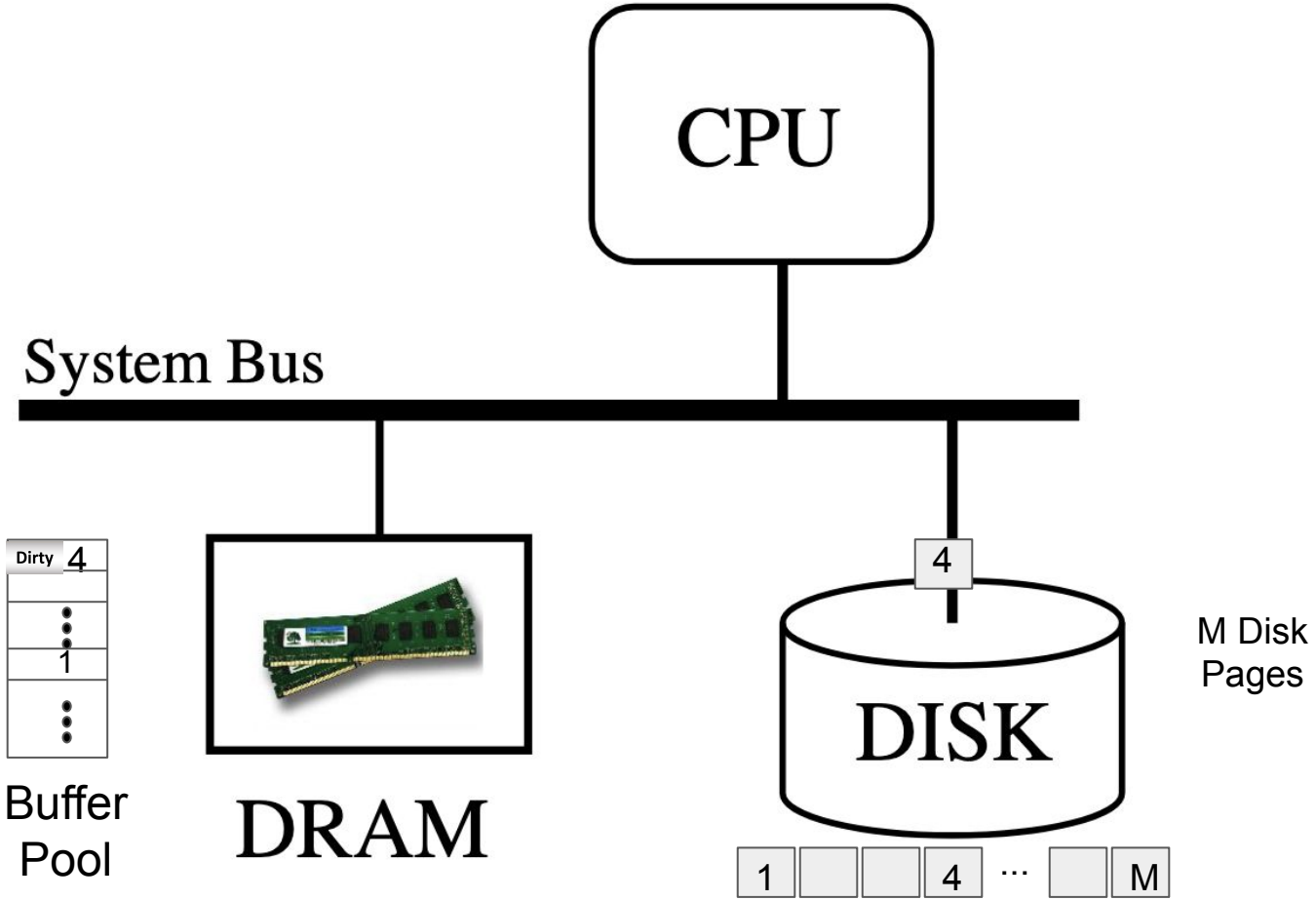
Disk Input/Output (I/O): Flush Page 4 to Disk



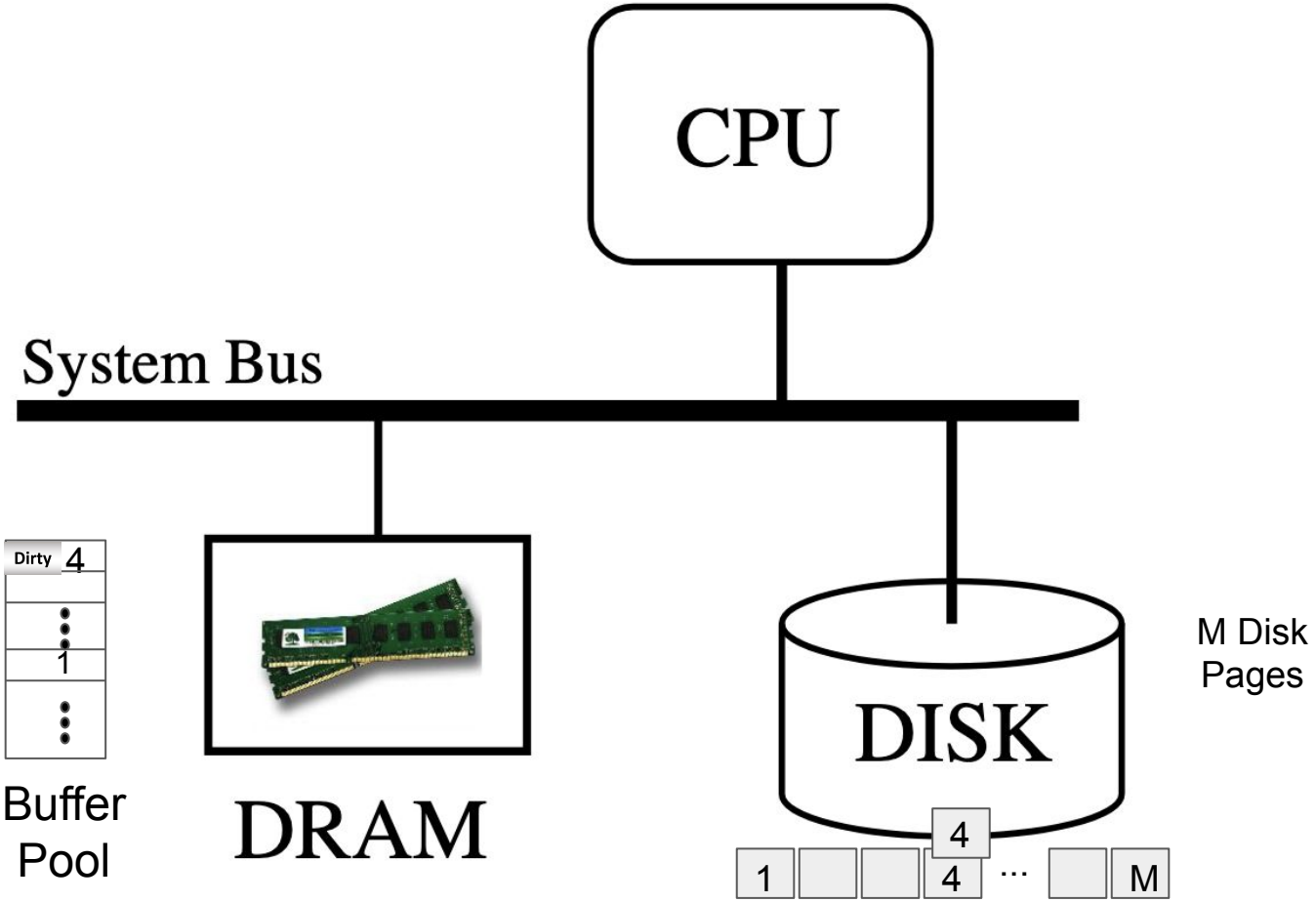
Disk Input/Output (I/O): Flush Page 4 to Disk



Disk Input/Output (I/O): Flush Page 4 to Disk

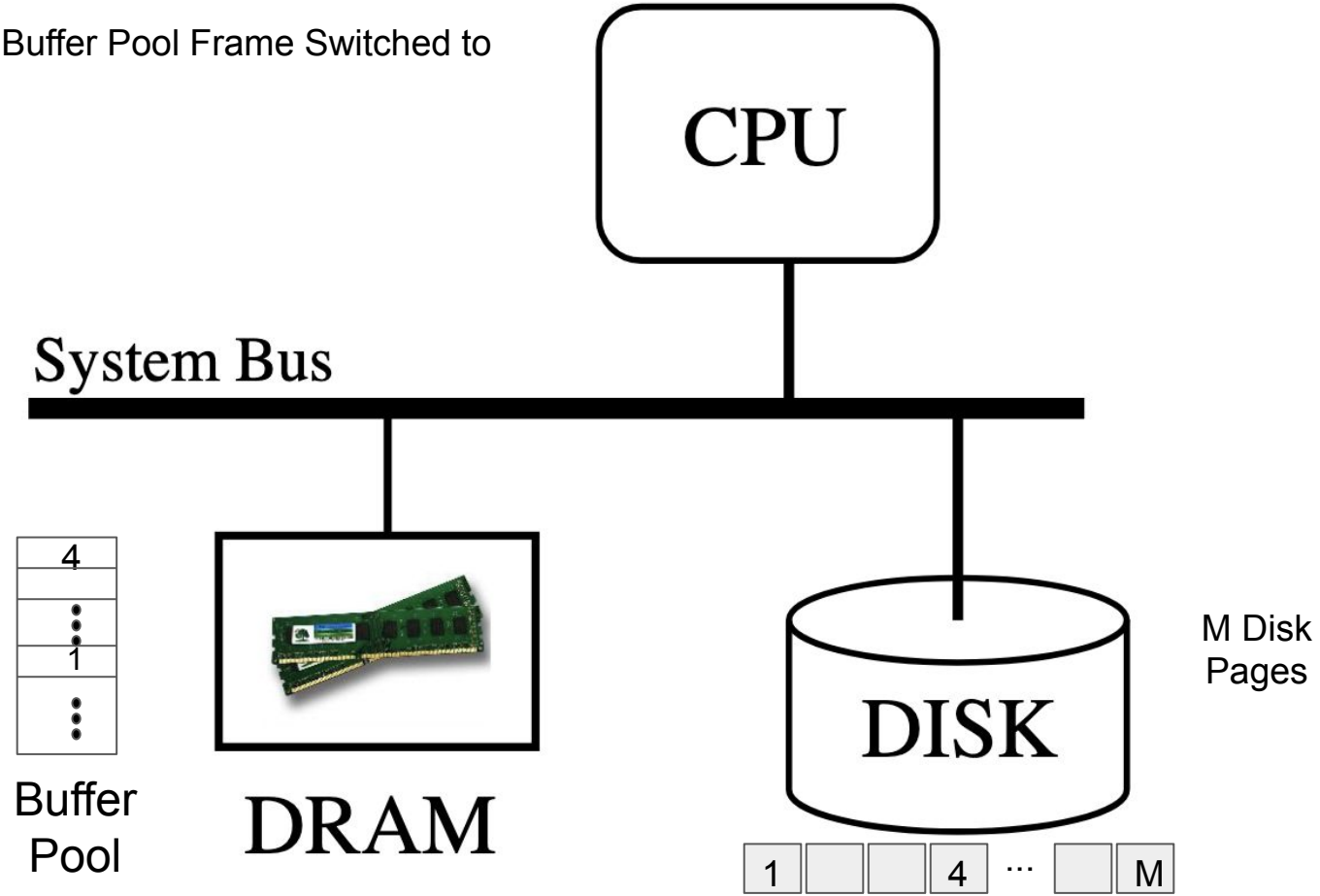


Disk Input/Output (I/O): Flush Page 4 to Disk



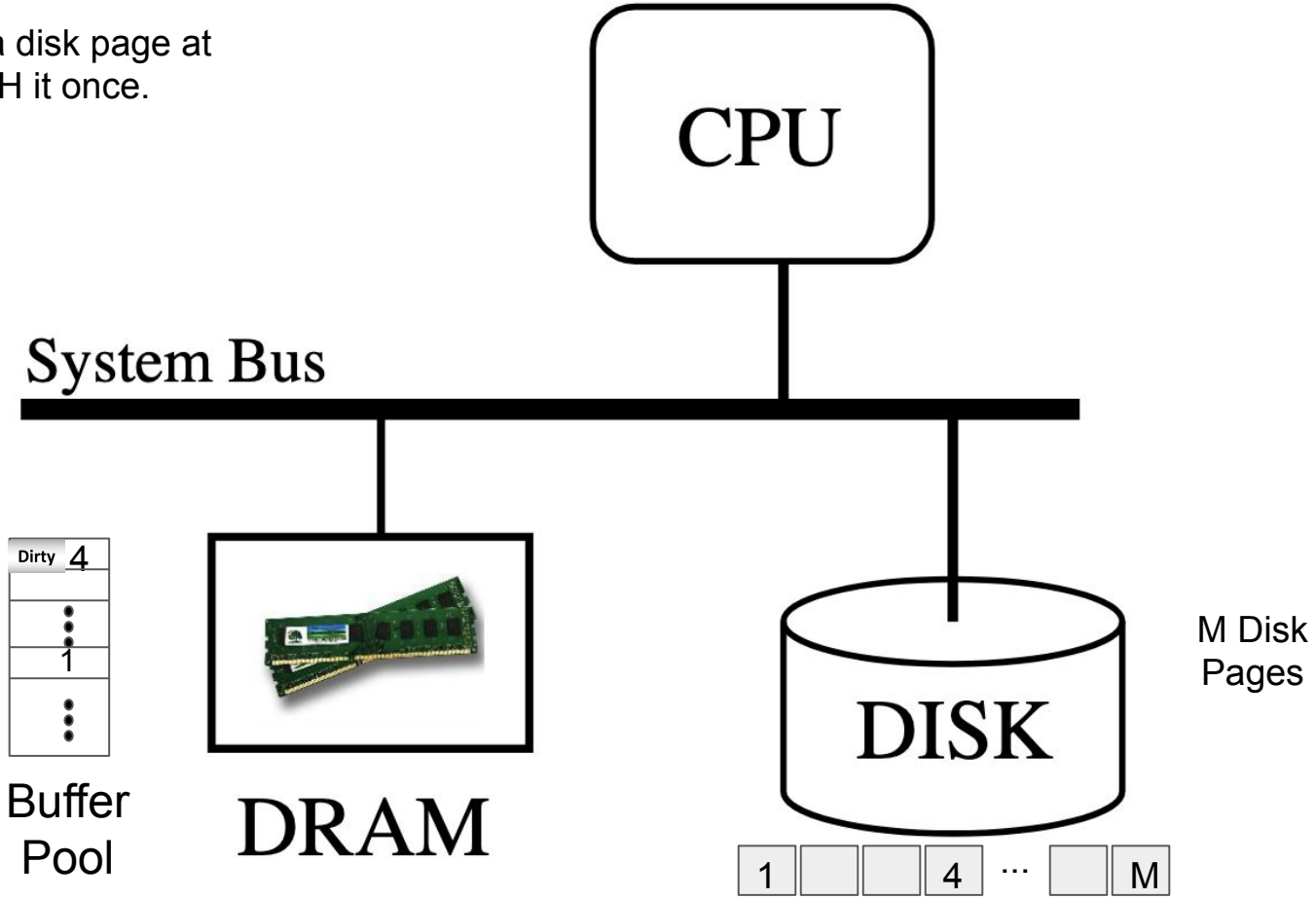
Disk Input/Output (I/O): Dirty becomes Clean

Flush complete: Dirty Buffer Pool Frame Switched to Clean.



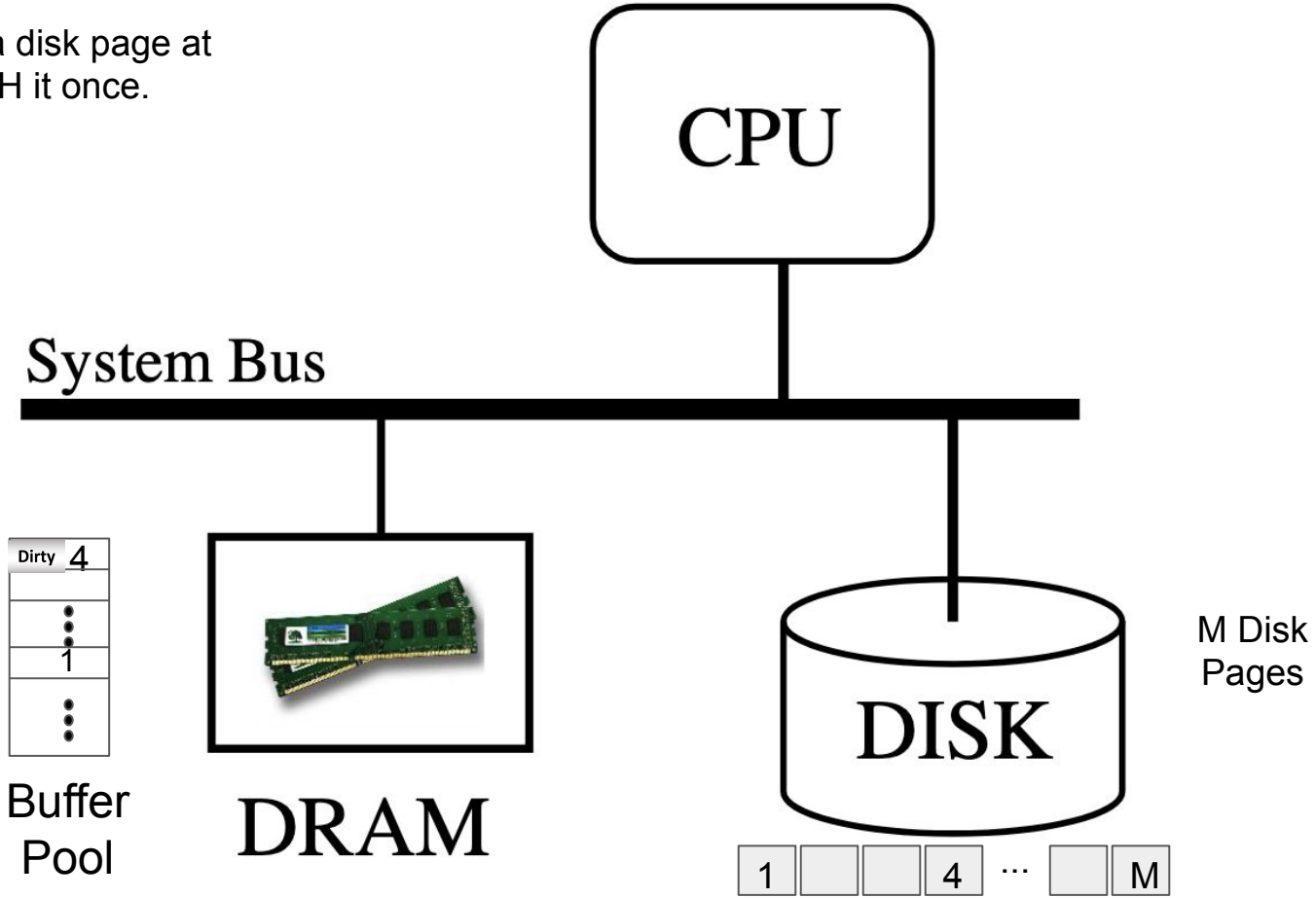
Disk Input/Output (I/O)

The DBMS may write a disk page at different times & FLUSH it once.

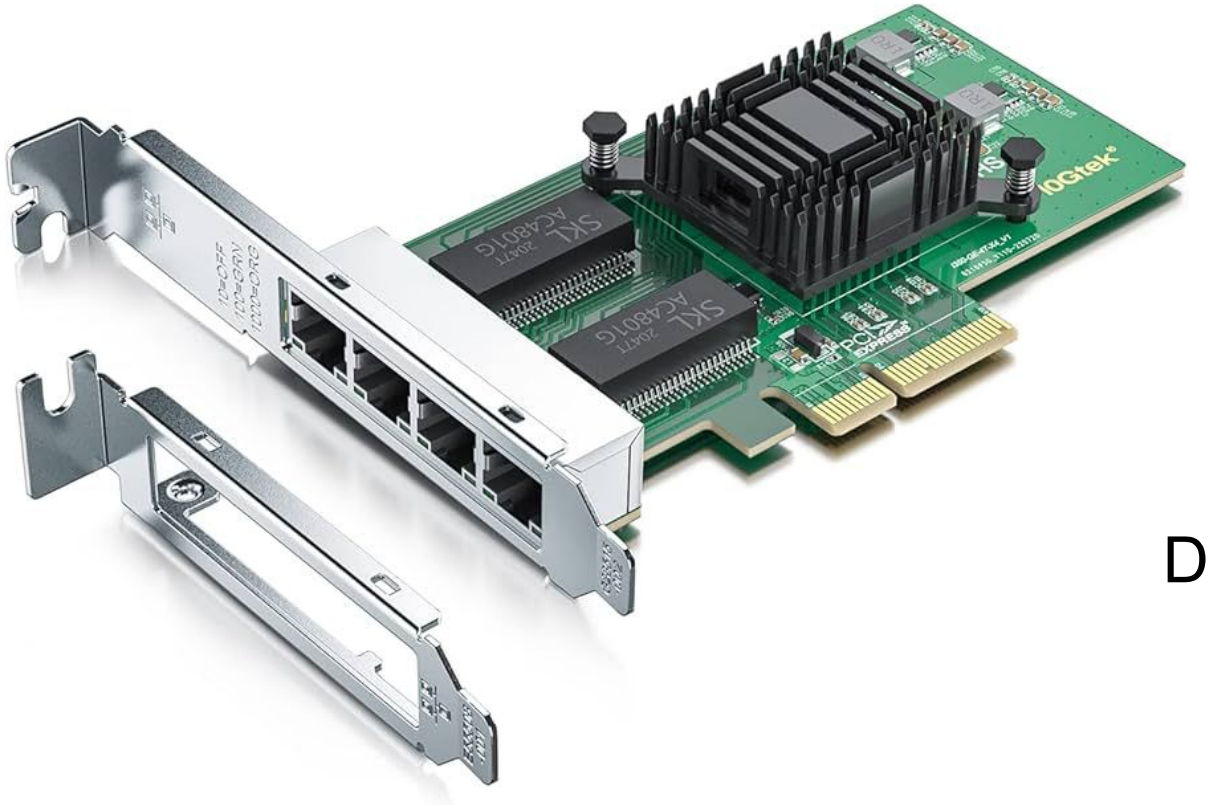


Disk Input/Output (I/O)

The DBMS may write a disk page at different times & FLUSH it once.



Network Interface Card, NIC



PCIe Gigabit Ethernet Server Adapter with Broadcom NetXtreme BCM5751 10/100/1000Mbps Gigabit Desktop PCI-E Network Card NIC

Visit the ULANSeN Store

4.4 ★★★★½ 122 ratings | Search this page

Amazon's Choice

in Internal Computer Networking Cards by ULANSeN

100+ bought in past month

$19.99

FREE Returns

Save up to 4% with business pricing. Sign up for a free Amazon Business account

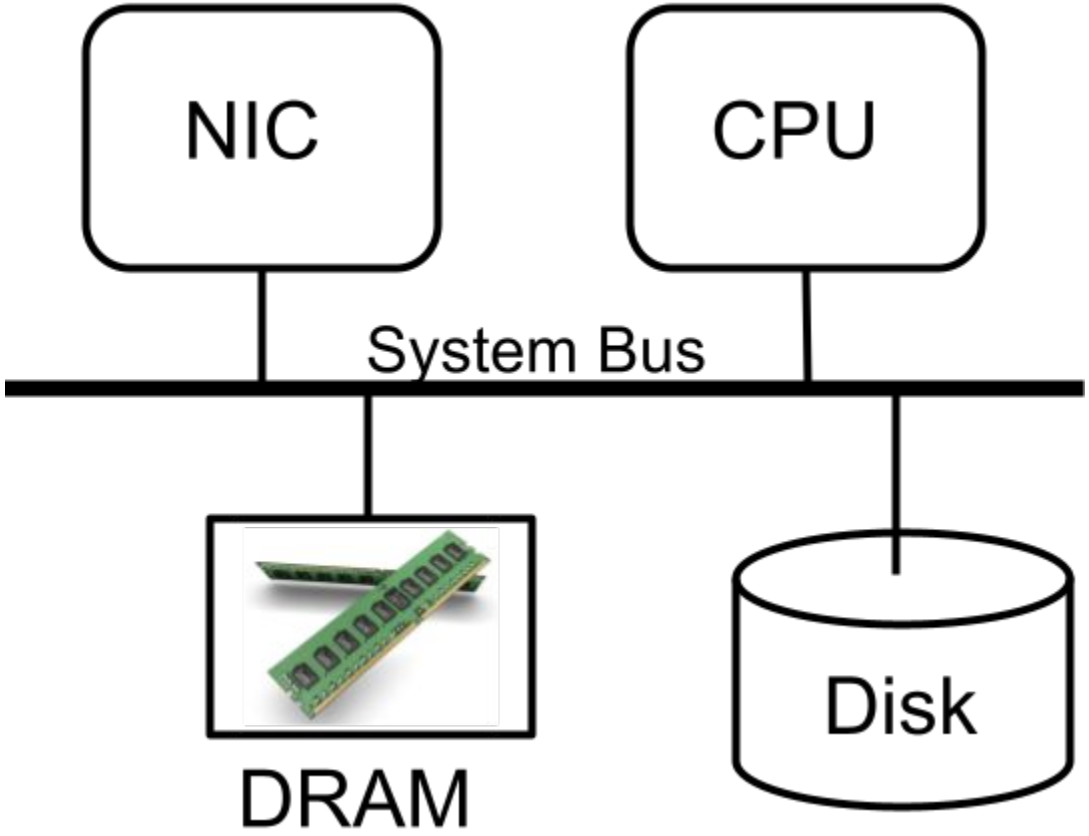
Get $60 off instantly: Pay $0.00 ~~$19.99~~ upon approval for the Amazon Store Card. No annual fee.

Brand	ULANSeN
Hardware Interface	PCI Express x1, Ethernet
Color	5751
Compatible Devices	Desktop
Product Dimensions	5"L x 0.8"W x 5"H

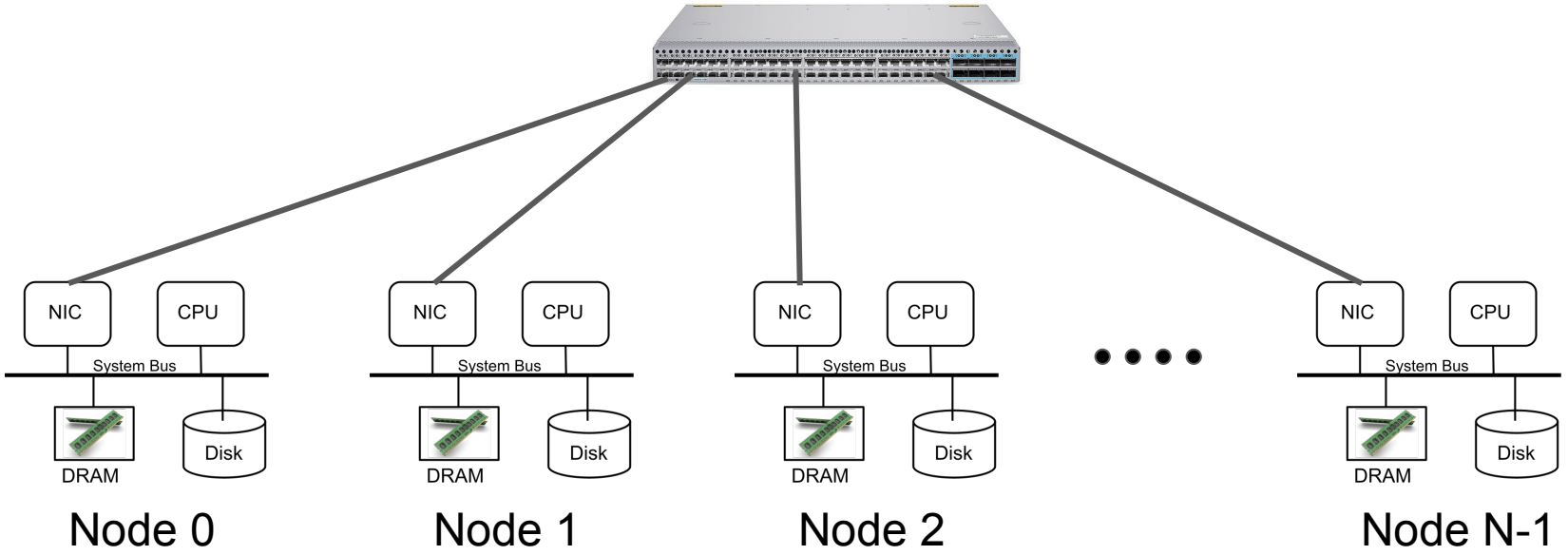
Captured Sept 5, 2024

Duplex?

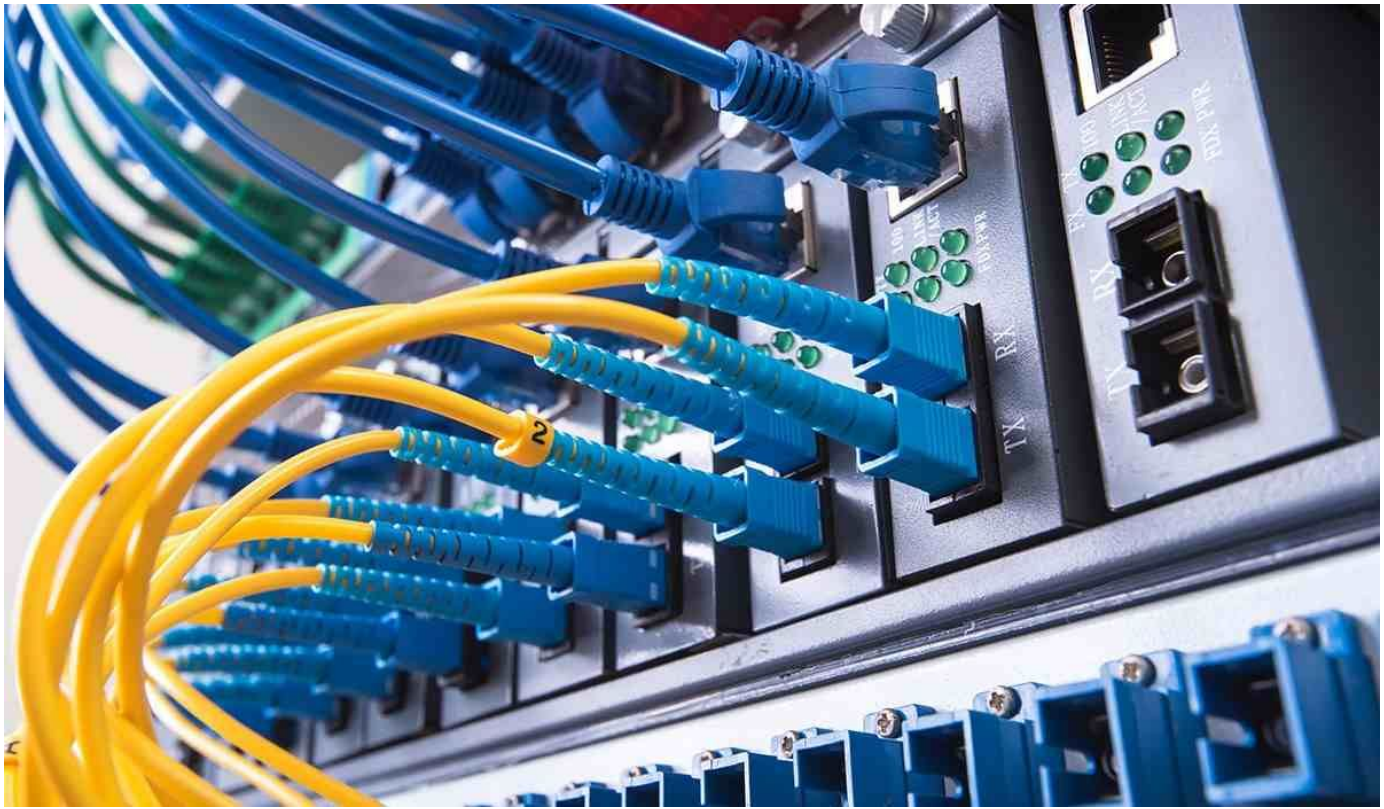
A Network Attached Node



A Shared-Nothing Architecture

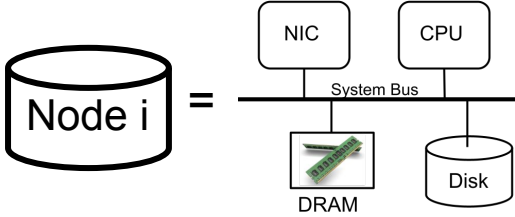


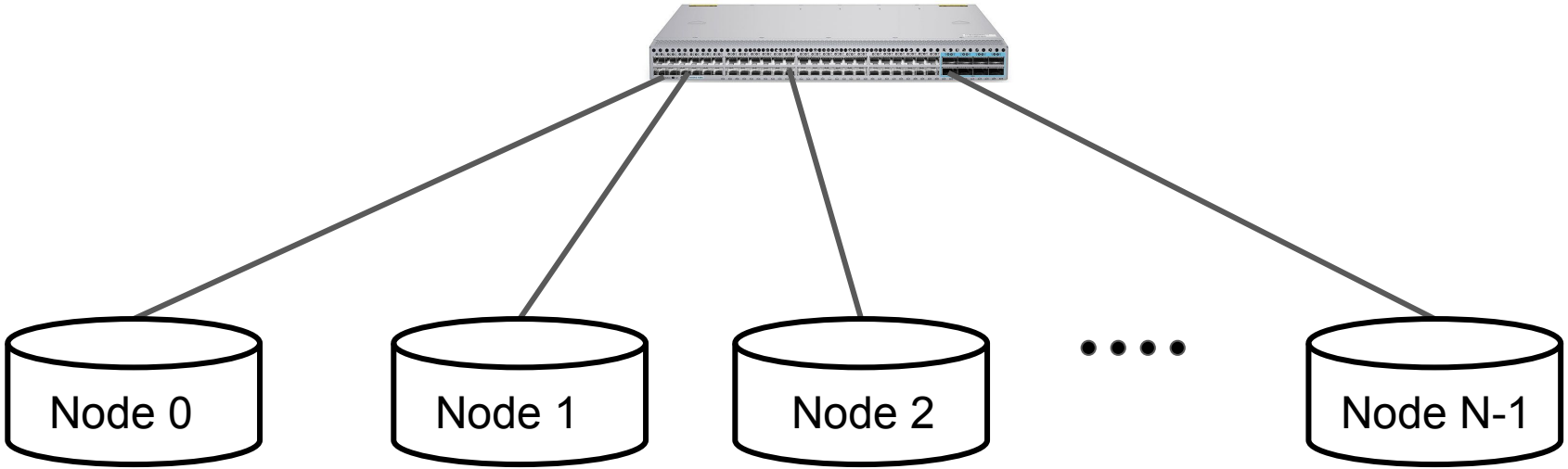
In Theory



In Practice

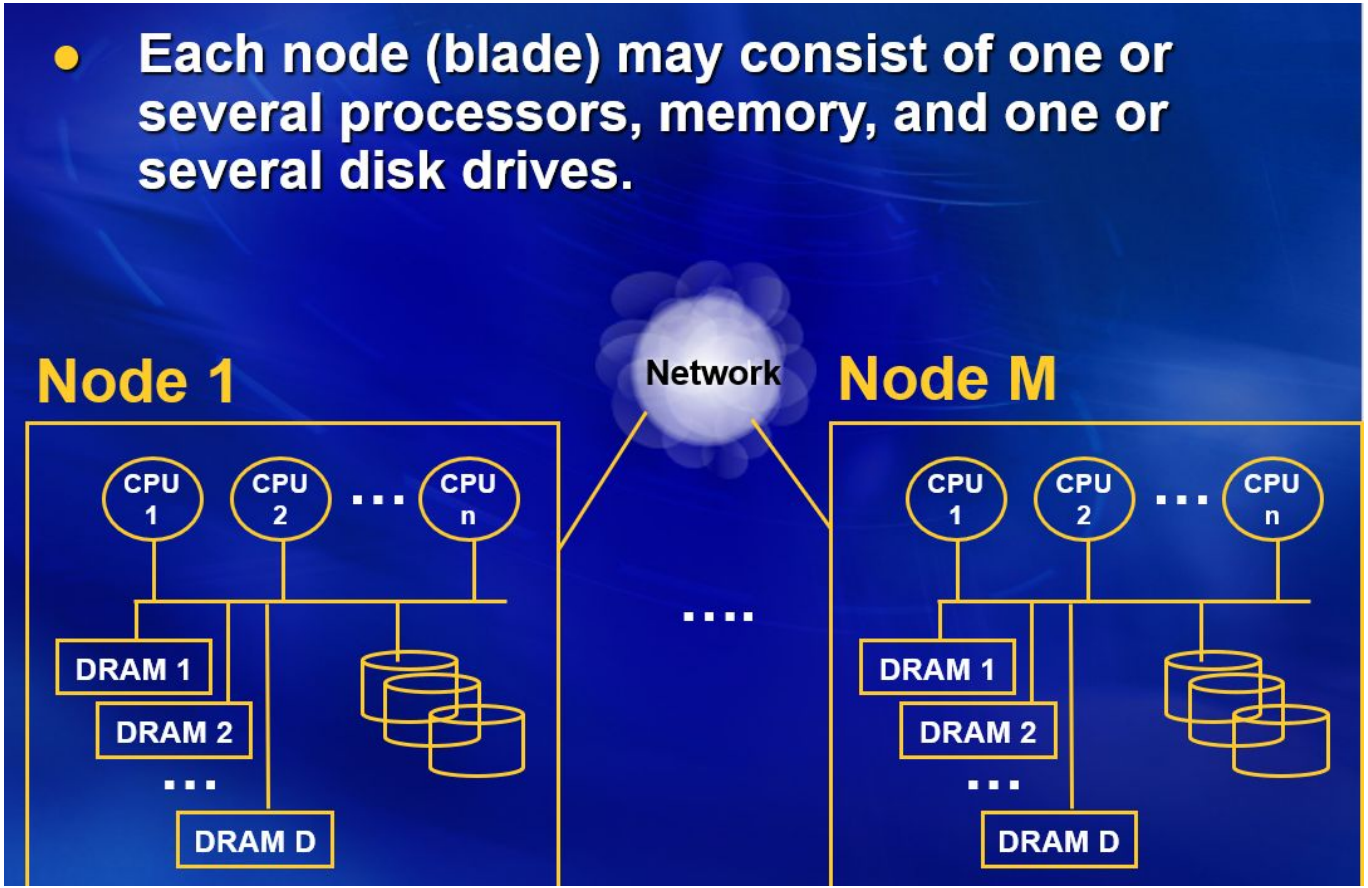
A Shared-Nothing Architecture





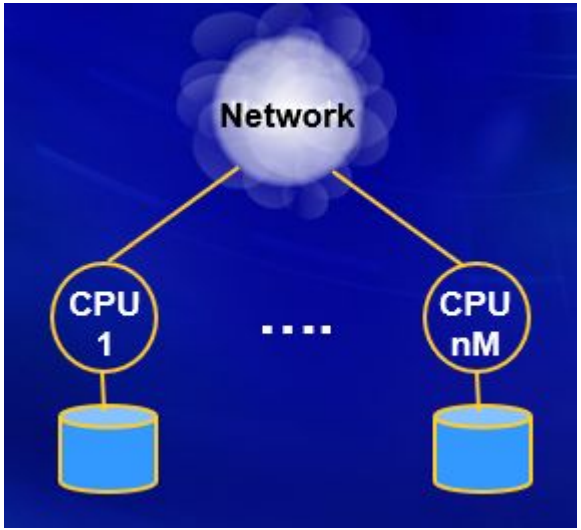
Virtual Nodes: Motivation

- Each node (blade) may consist of one or several processors, memory, and one or several disk drives.



Virtual Nodes

Partition resources to construct logical nodes. With an 8 core processor, construct eight logical nodes each with one processor, a fraction of memory, and one disk drive.



Why use multiple nodes?

Speedup: Perform a complex query/update faster. Ideally, it should become more than N times faster with N nodes. Example update/query:

1. Give all employees a 10% raise.
2. What is the average salary of employees with 2 or more children working in different departments.

Scaleup: Maintain the same speed for a complex query/update as the database grows by increasing the number of nodes. Example:

- Consider the above query. If the database grows 10x, maintain the same speed by increasing the number of nodes 10x.

How?

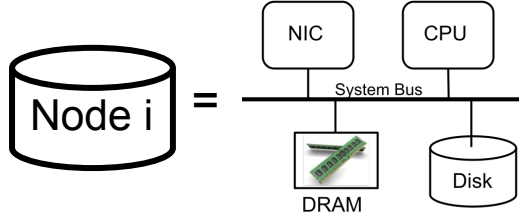
Design philosophy:

- Divide and conquer by partitioning the data.
- Use decentralized algorithms.
- Minimize coordination.

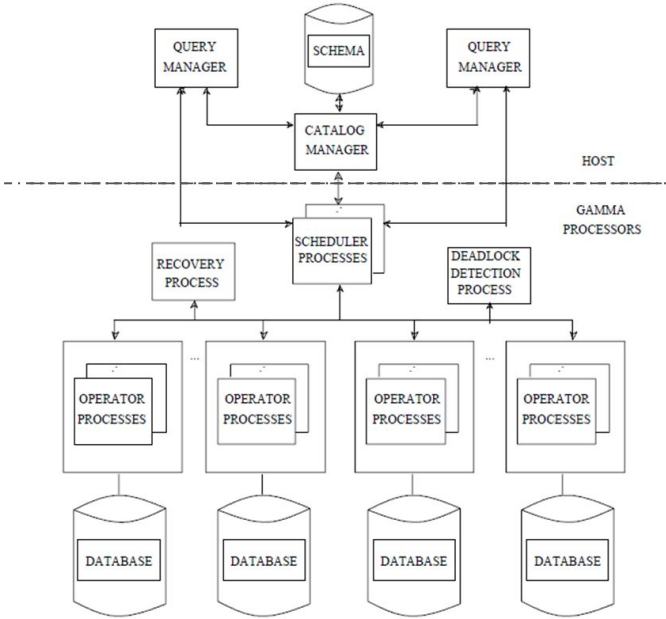
Challenges:

- Nodes fail.
- Latency associated with network communication.
- Design and implementation of decentralized algorithms is non-trivial.

Gamma Architecture

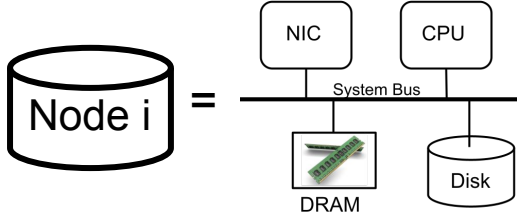


- Each node stores its fragment on its local disk drive
- Each node may build a B+-tree (clustered/non-clustered) and hash index on its fragment of a table.
- Each node has its own concurrency control and crash recovery protocol.



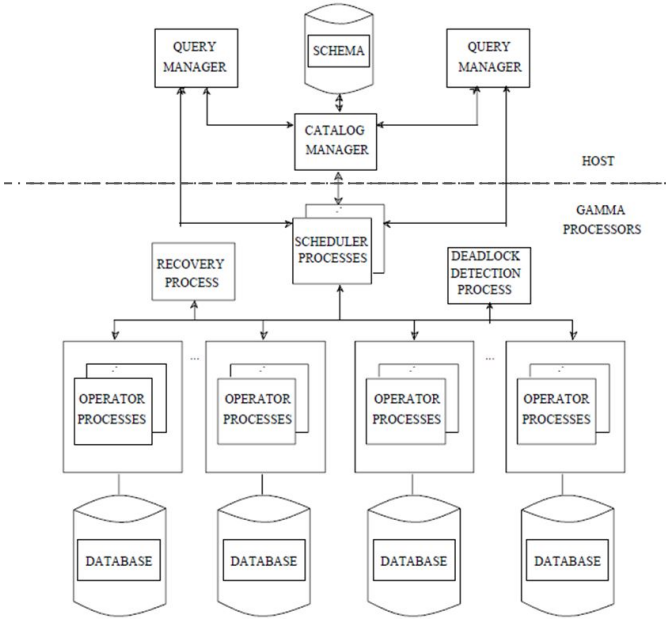

```
SELECT    *
FROM      Emp
WHERE     salary=60K
```

Gamma Architecture



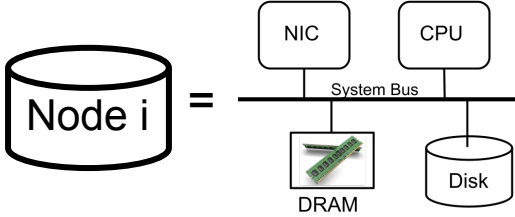
- Each node stores its fragment on its local disk drive
- Each node may sh index on its fragment of a
- Each node has its own concurrency control and crash recovery protocol.

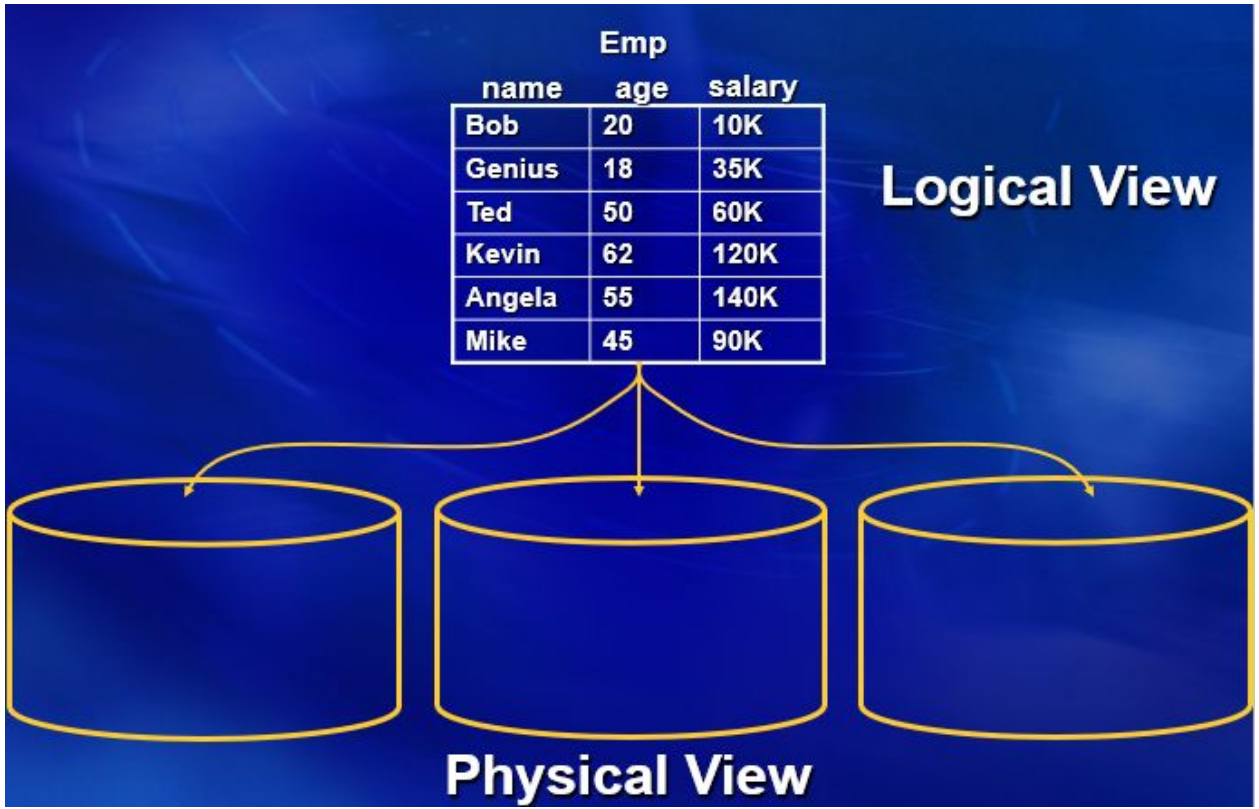
SQL to hide parallelism: User is not aware of parallel processing.



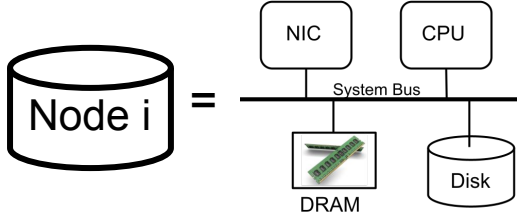

```
SELECT    *
FROM      Emp
WHERE     salary=60K
```

Horizontal Declustering/Partitioning/Sharding





Horizontal Declustering: How?



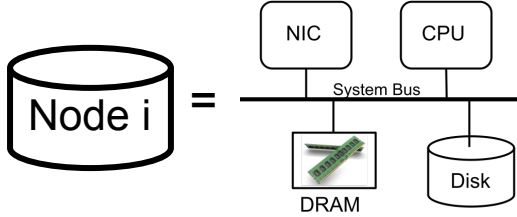
No partitioning attribute: Random and Round-robin.

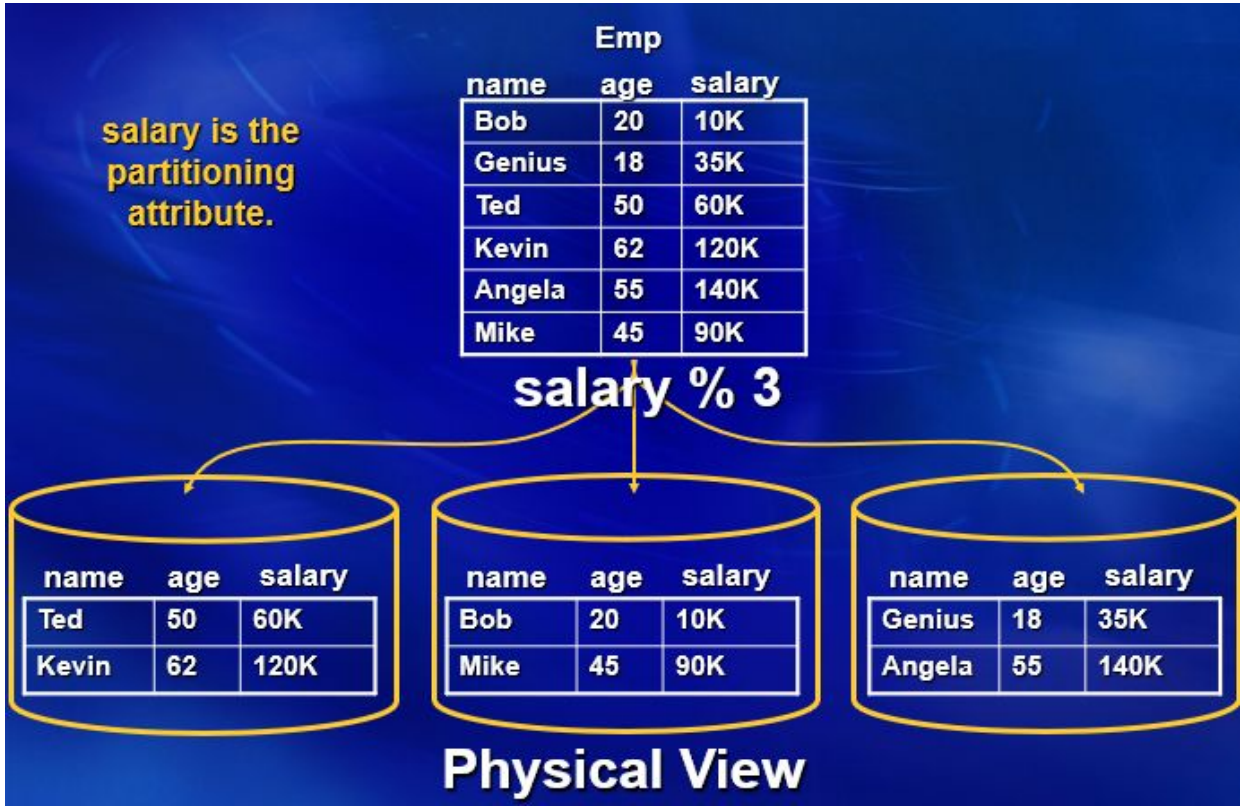
Single attribute declustering strategy:

1. Hash
2. Range

Note: The administrator must select an attribute as the partitioning attribute.

Hash Declustering/Partitioning/Sharding





What is the MOD (%) Function?

F(x) = x MOD N = x % N

- It is the remainder of x divided by N. Assume N=10:
  - F(12) = 12 % N = 2
  - F(10) = 10 % N = 0
  - F(1) = 1 % N = 1
  - F(101) = 101 % N = 1
  - F(20,511) = 20,511 % N = 1

- x MOD N maps a value x to a number between 0 and N-1.
- It is simple and perhaps the most popular hash functions out there.

Hash Declustering

Selections with equality predicates referencing the partitioning attribute are directed to a single node:

- Retrieve Employees with a 60K salary

```
SELECT *
FROM   Emp
WHERE salary=60K
```

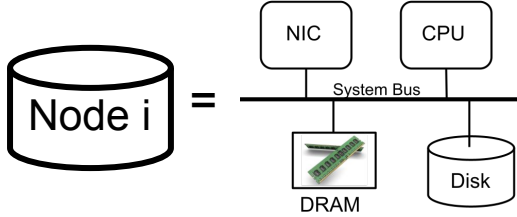
Equality predicates referencing a non-partitioning attribute and range predicates are directed to all nodes:

- Retrieve Employees who are 20 years old
- Fetch Employees with a salary lower than 50K

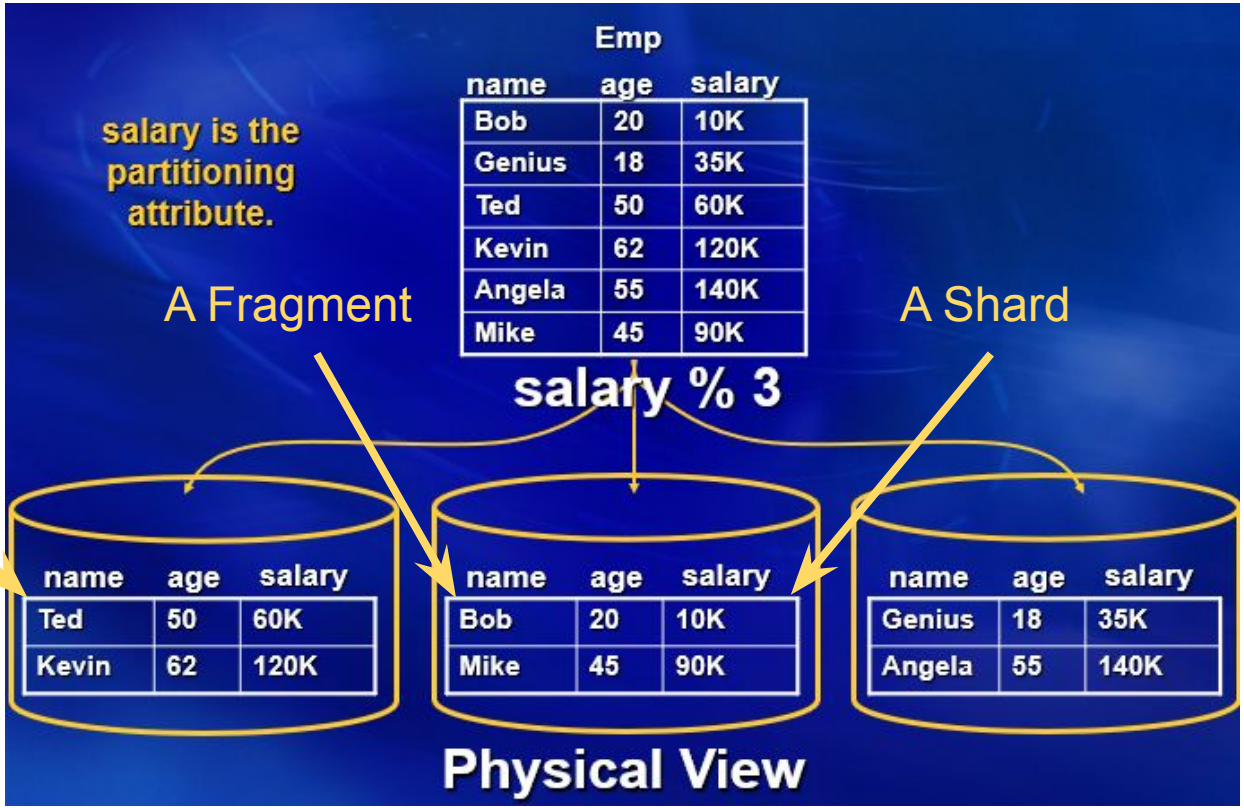
```
SELECT *
FROM   Emp
WHERE age=20
```

```
SELECT *
FROM   Emp
WHERE salary < 50K
```

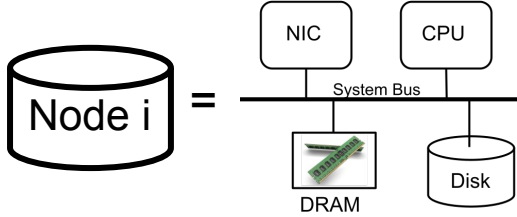
A Fragment/Shard



A Fragment



Range Declustering/Partitioning/Sharding



salary is the partitioning attribute.

Emp

name	age	salary
Bob	20	10K
Genius	18	35K
Ted	50	60K
Kevin	62	120K
Angela	55	140K
Mike	45	90K

0-50K

name	age	salary
Bob	20	10K
Genius	18	35K

51K-100K

name	age	salary
Ted	50	60K
Mike	45	90K

101K-∞

name	age	salary
Kevin	62	120K
Angela	55	140K

Physical View

Range Declustering

Equality and range predicates referencing the partitioning attribute are directed to a subset of nodes:

- Retrieve Employees with a 60K salary
- Fetch Employees with a salary lower than 50K

```
SELECT *
FROM   Emp
WHERE salary=60K
```

```
SELECT *
FROM   Emp
WHERE salary < 50K
```

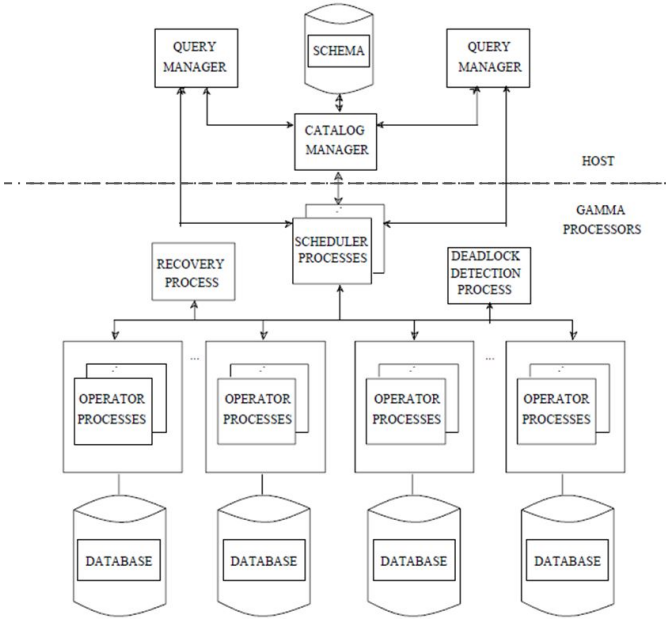
Predicates referencing a non-partitioning attribute are directed to all nodes:

- Retrieve Employees who are 20 years old

```
SELECT *
FROM   Emp
WHERE age=20
```

Architecture

- Each node stores its fragment on its local disk drive
- Each node may build a B+-tree (clustered/non-clustered) and hash index on its fragment of a table.
- Each node has its own concurrency control and crash recovery protocol.

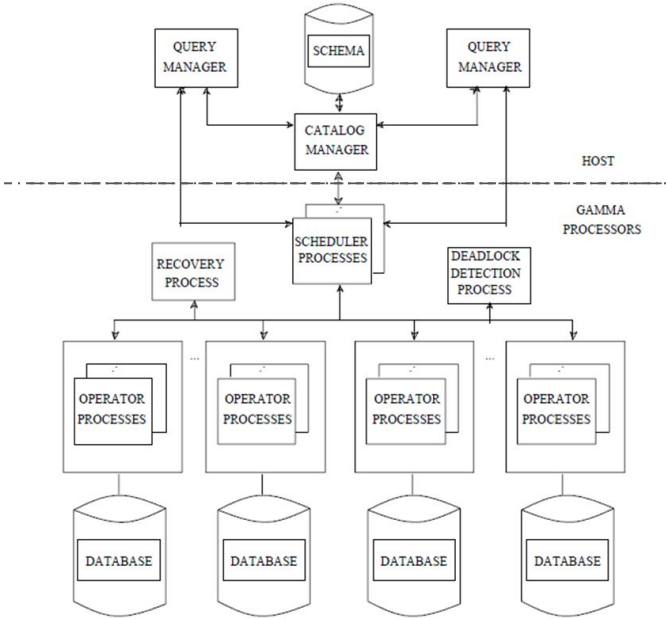



```
SELECT    *
FROM      Emp
WHERE     salary=60K
```

Architecture

Catalog Manager

The function of the Catalog Manager is to act as a central repository of all conceptual and internal schema information for each database. The schema information is loaded into memory when a database is first opened. Since multiple users may have the same database open at once and since each user may reside on a machine other than the one on which the Catalog Manager is executing, the Catalog Manager is responsible for insuring consistency among the copies cached by each user.




```
SELECT    *
FROM      Emp
WHERE     salary=60K
```

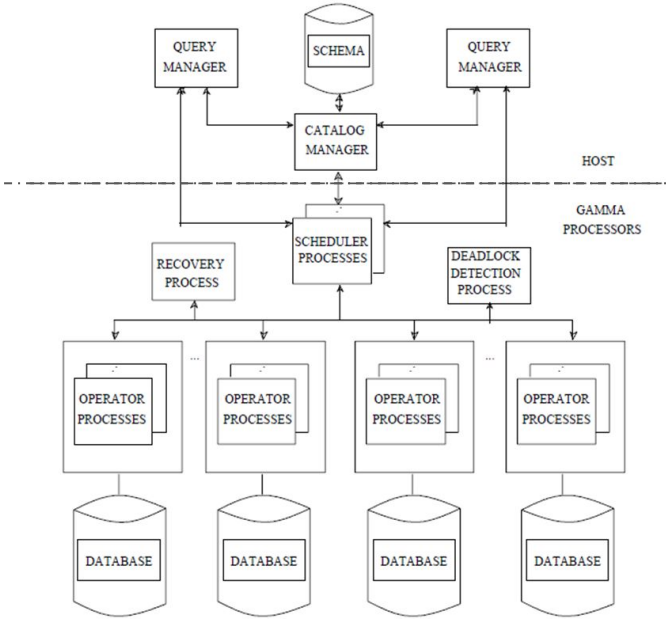
Architecture

- Each node stores its fragment on its local disk drive

Query Manager

One query manager process is associated with each active Gamma user. The query manager is responsible for caching schema information locally, providing an interface for ad-hoc queries using gdl (our variant of Quel [STON76]), query parsing, optimization, and compilation.

- Each node has its own concurrency control and crash recovery protocol.




```
SELECT   *
FROM     Emp
WHERE    salary=60K
```

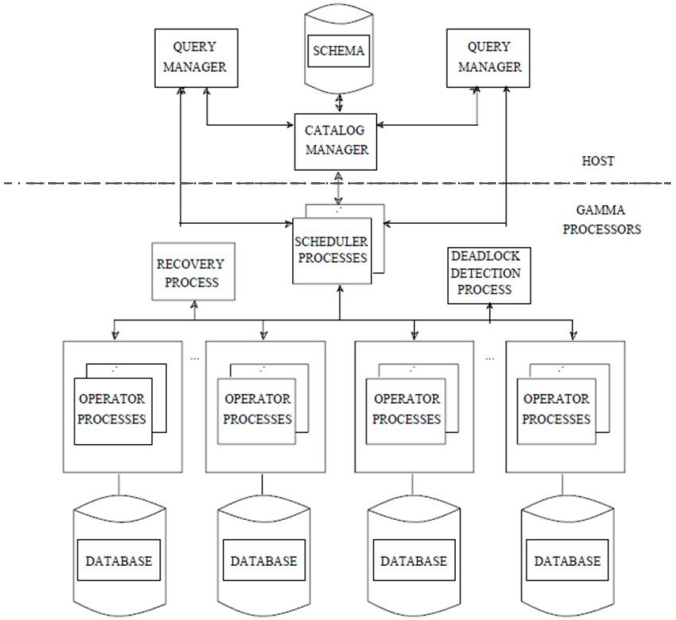
Architecture

- Each node stores its fragment on its local disk drive

Query Manager

One query manager process is associated with each active Gamma user. The query manager is responsible for caching schema information locally, providing an interface for ad-hoc queries using gdl (our variant of Quel [STON76]), query parsing, optimization, and compilation.

- Each node has its own concurrency control and crash recovery protocol.

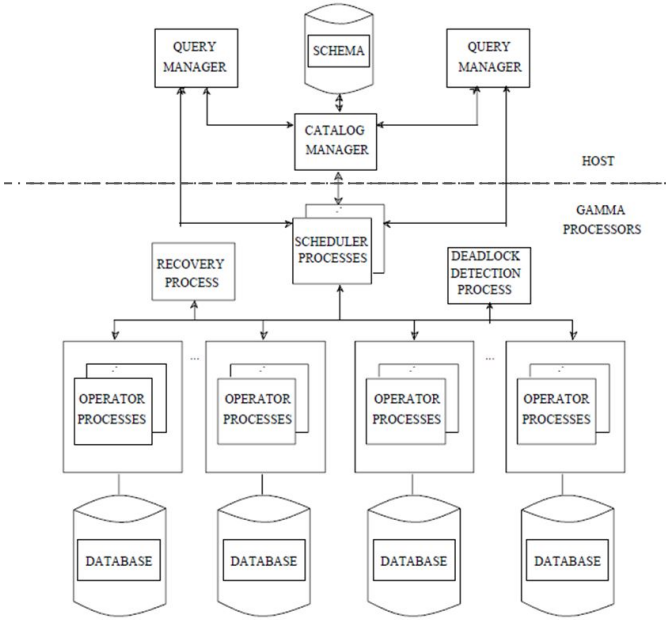


in the system. In the case of a single site query, the query is sent directly by the QM to the appropriate processor for execution. In the case of a multiple site query, the optimizer establishes a connection to an idle scheduler process through a centralized dispatcher process. The dispatcher process, by controlling the number of active schedulers, implements a simple load control mechanism. Once it has established a connection with a scheduler process, the QM sends the compiled query to the scheduler process and waits for the query to complete execution. The scheduler process, in turn, activates operator processes at each query processor selected to execute the operator. Finally, the QM reads the results of the query and returns them through the ad-hoc query interface to the user or through the embedded query interface to the program from which the query was initiated.

Architecture

Scheduler Processes

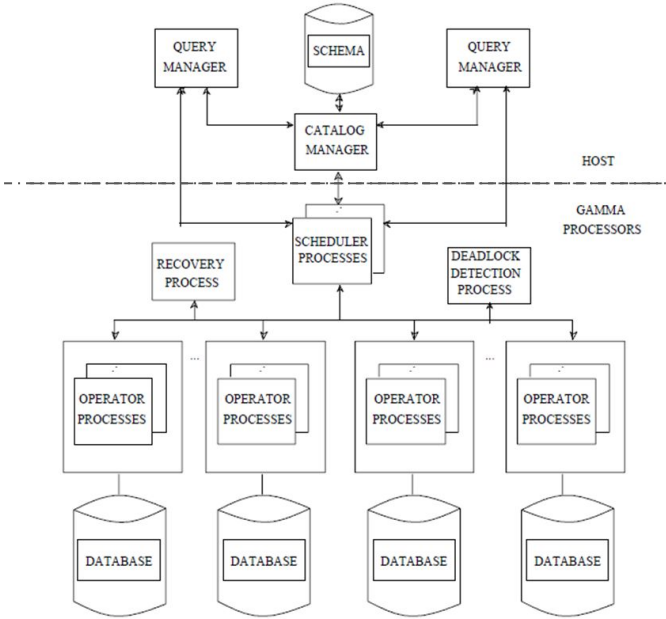
While executing, each multisite query is controlled by a scheduler process. This process is responsible for activating the Operator Processes used to execute the nodes of a compiled query tree. Scheduler processes can be run on any processor, insuring that no processor becomes a bottleneck. In practice, however, scheduler processes consume almost no resources and it is possible to run a large number of them on a single processor. A centralized dispatching process is used to assign scheduler processes to queries. Those queries that the optimizer can detect to be single-site queries are sent directly to the appropriate node for execution, by-passing the scheduling process.




```
SELECT    *
FROM      Emp
WHERE     salary=60K
```

Operator Process

For each operator in a query tree, at least one Operator Process is employed at each processor participating in the execution of the operator. These operators are primed at system initialization time in order to avoid the overhead of starting processes at query execution time (additional processes can be forked as needed). The structure of an operator process and the mapping of relational operators to operator processes is discussed in more detail below. When a scheduler wishes to start a new operator on a node, it sends a request to a special communications port known as the "new task" port. When a request is received on this port, an idle operator process is assigned to the request and the communications port of this operator process is returned to the requesting scheduler process.




```
SELECT    *
FROM      Emp
WHERE     salary=60K
```

Software Architecture

- Processes executing on one node share memory - identical to today's threads.
- At initialization time, a node starts a fixed number of threads (processes).
- All threads listen on a well defined socket, waiting for the Scheduler to dispatch work to them.
- A message contains the identity that the operator should assume:
  - A switch statement enables a thread to become a select, a project, hash-join build, hash-join probe, etc.
  - The message specifies the role of the thread.

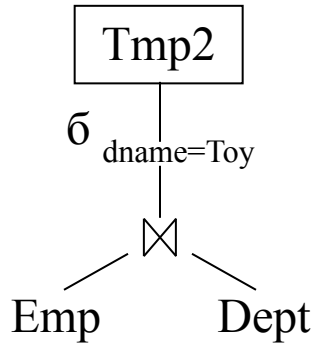
Operator Process

For each operator in a query tree, at least one Operator Process is employed at each processor participating in the execution of the operator. These operators are primed at system initialization time in order to avoid the overhead of starting processes at query execution time (additional processes can be forked as needed). The structure of an operator process and the mapping of relational operators to operator processes is discussed in more detail below. When a scheduler wishes to start a new operator on a node, it sends a request to a special communications port known as the "new task" port. When a request is received on this port, an idle operator process is assigned to the request and the communications port of this operator process is returned to the requesting scheduler process.

Pipelined Query Execution

1. Initiate a process for Store into TMP 2 operator.
2. Initiate a process for the select operator.
3. Initiate a Join process.
4. Read the Emp records and forward to the join process to build a hash table.
5. Read the Dept table and forward to the join process to probe the hash table. Forward each joining record to the Select process of Step 2.
6. Select operator prunes those records with dname=Toy and forwards them to the Store TMP2 process of Step 1.
7. TMP2 process writes qualifying records to disk.

- $TMP2 \leftarrow \sigma_{dname=Toy}(Emp \bowtie Dept)$

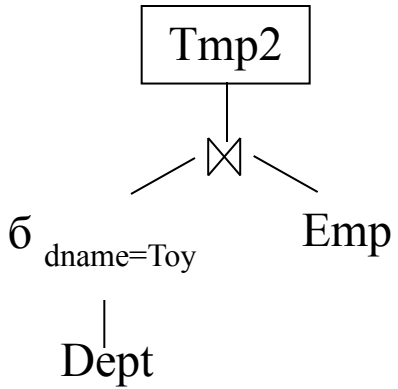



```
SELECT * into TMP2
FROM Emp e, Dept d
WHERE e.dno = d.dno and
d.dname = 'Toy'
```

Pipelined Query Execution

- TMP2 ← $\sigma_{dname=Toy}(Emp \bowtie Dept)$

1. Initiate a process for the Store into TMP 2 operator.
2. Initiate a process for the Join operator.
3. Initiate a process for the select operator that reads the Dept table and produces all the records with dname = Toy, forwarding them to the Join operator of Step 2.
4. The Join Build operator builds a hash table on dname of its received records.
5. Read the Emp record and forward to the Join operator. This operator probes the hash table for the qualifying record, forwarding them to the Store operator of Step 1.



Important Details

- The input and output of an operator is a stream of records.
- The select operator reads the records of a fragment and produce those satisfying a selection predicate as its output stream.
  - It is possible that no selection predicate is provided to the select operator. In this case, it becomes a simple ***scan*** that produces all records of a fragment.
- The join operator consists of two distinct steps.

Hash-Join Operator

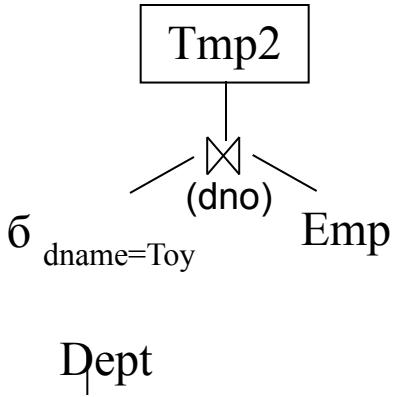
Consists of 2 distinct and sequential steps:

1. Build: Constructs a hash table on the join attribute.
2. Probe: Look up the hash table for a joining record.

In our example:

1. Build: Construct a hash table on the dno of Dept records with dname=Toy.
2. Probe: Look up the hash table with the Emp.dno of each Emp record.

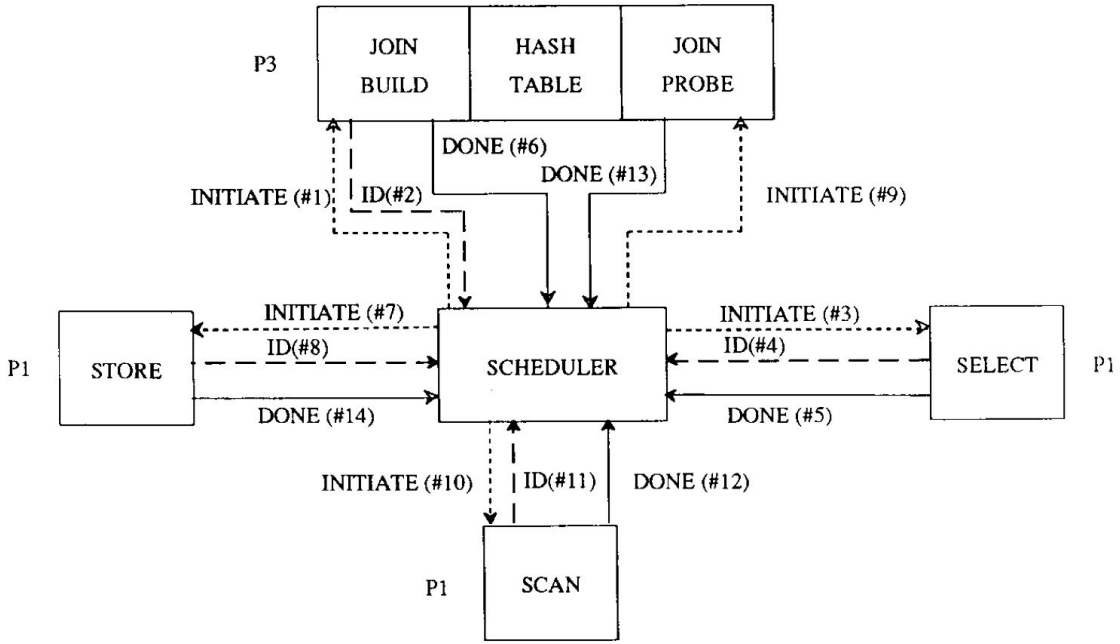
Sequential means the two steps follow one another. Probe begins after Build completes.

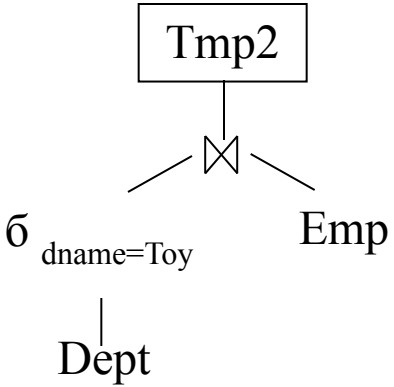


How does an operator know where to send its stream of records?

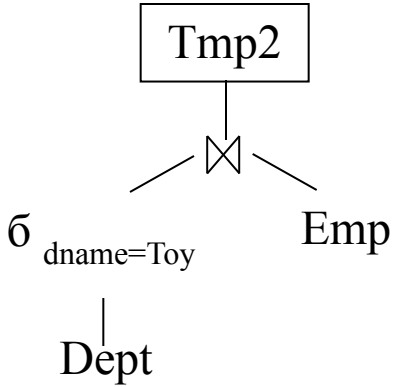
Scheduler Informs Them!

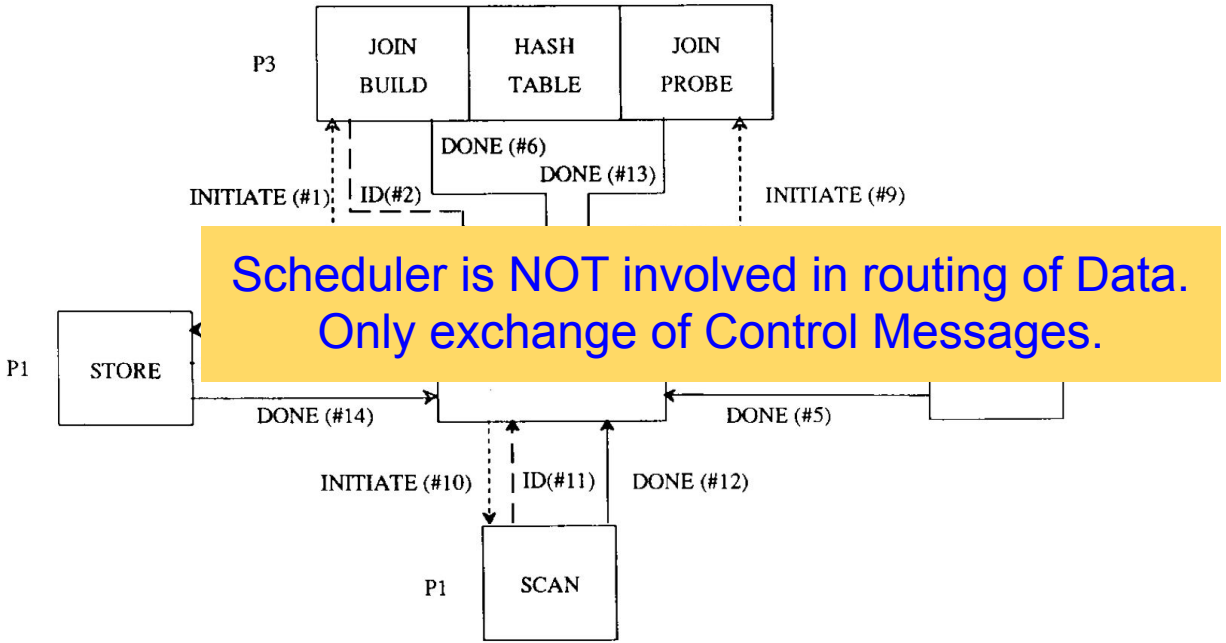
Scheduler controls and coordinates:





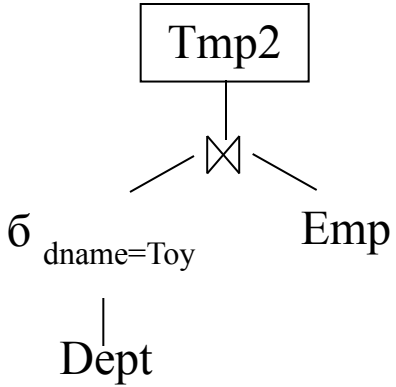
Scheduler Coordinates Flow of Data

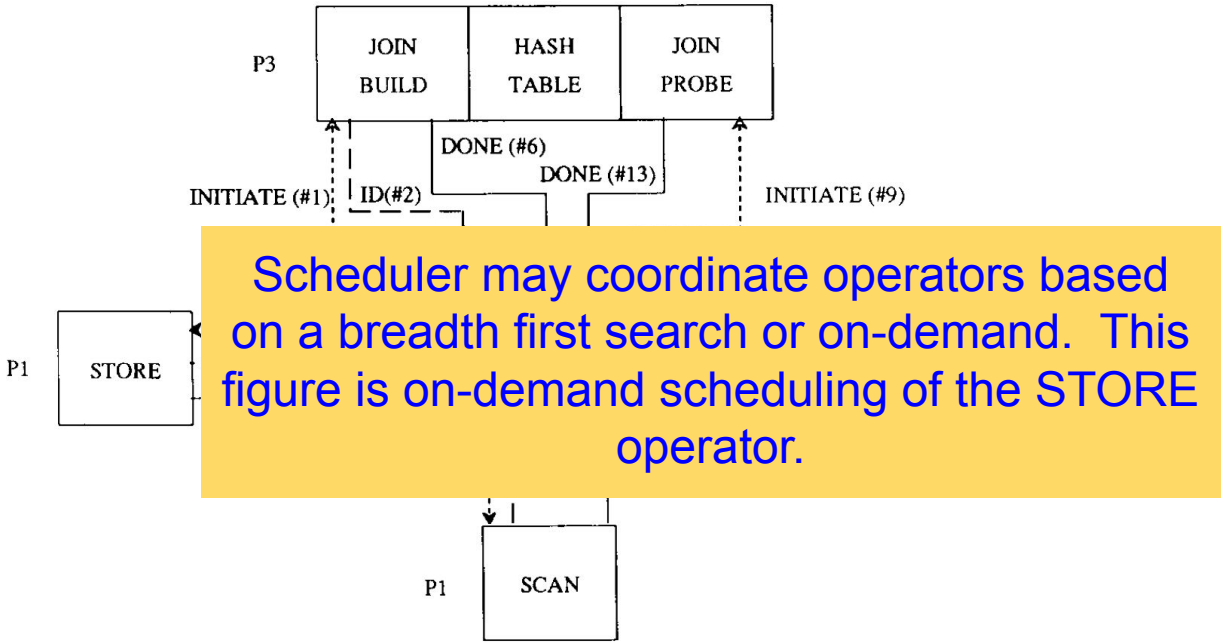




Scheduler is NOT involved in routing of Data.
Only exchange of Control Messages.

Scheduler Coordinates Flow of Data





Scheduler may coordinate operators based on a breadth first search or on-demand. This figure is on-demand scheduling of the STORE operator.

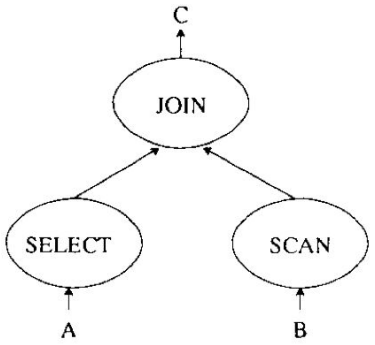
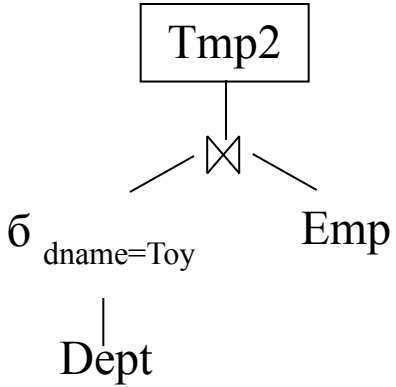
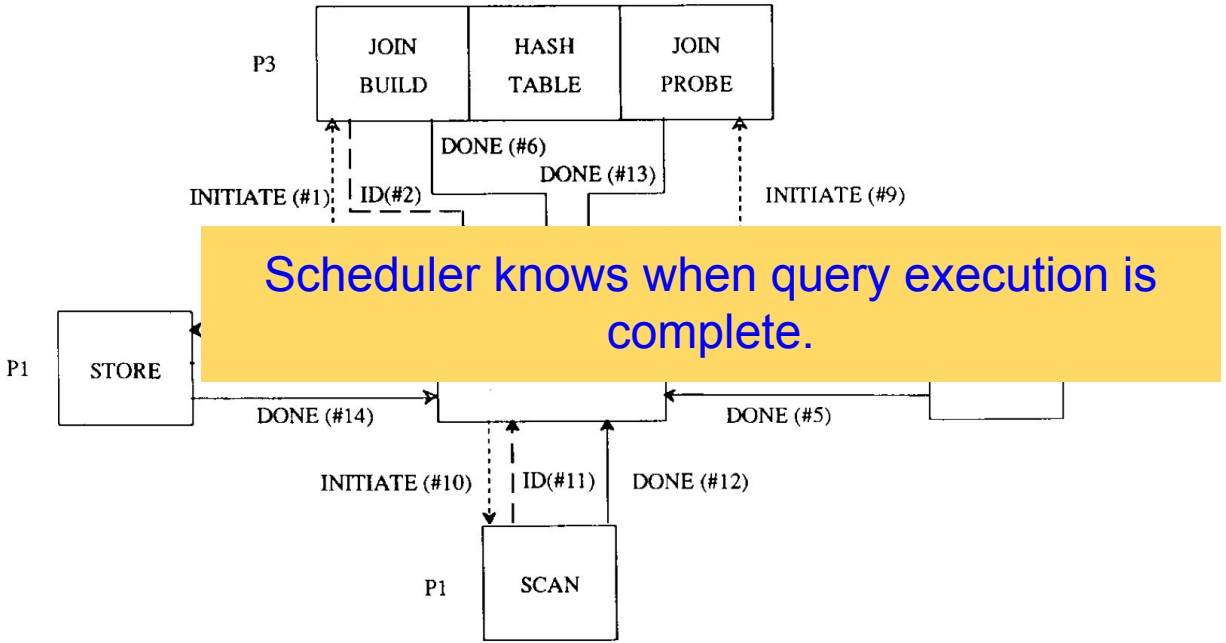


Fig. 5.

Scheduler Terminates Query





Scheduler knows when query execution is complete.

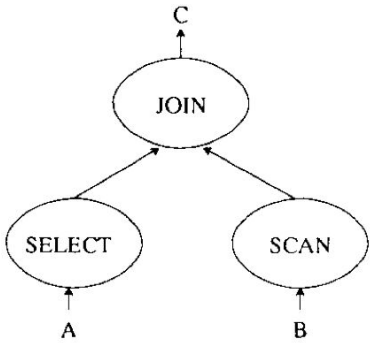
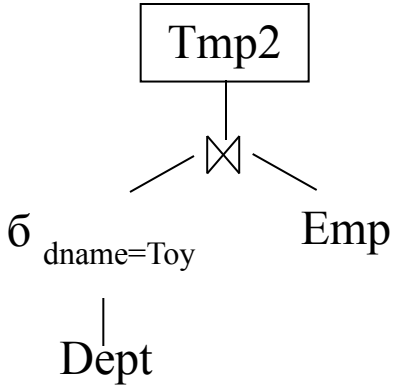
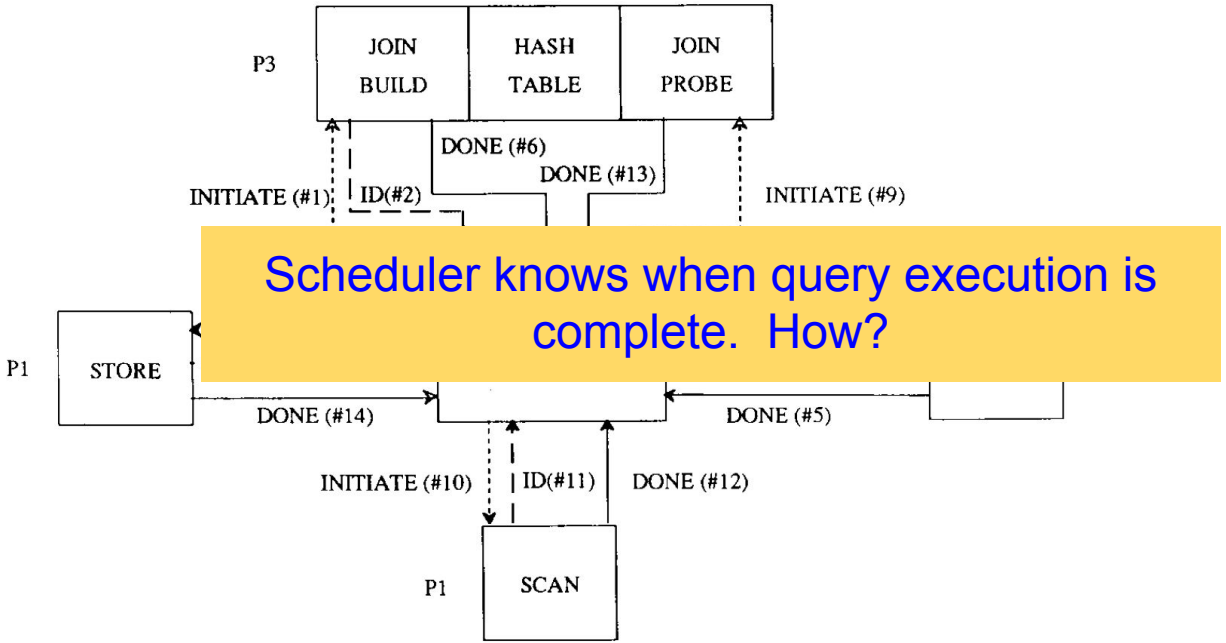


Fig. 5.

Scheduler Coordinates Flow of Data





Scheduler knows when query execution is complete. How?

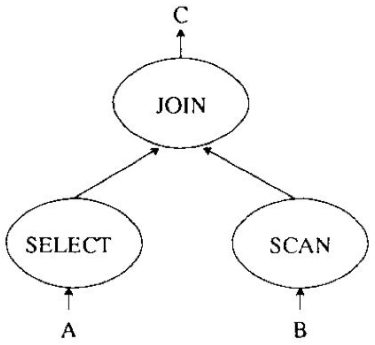
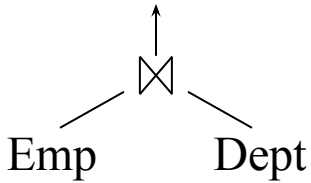


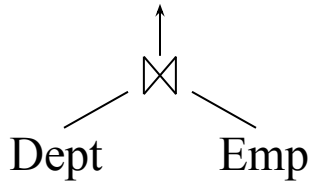
Fig. 5.

Hash-Join: Not Enough Memory



What happens if the table in the build phase does not fit in memory? Example: Emp table is 100 Gigabytes and the amount of memory is 16 Gigabytes.

Hash-Join: Not Enough Memory



What happens if the table in the build phase does not fit in memory? Example: Emp table is 100 Gigabytes and the amount of memory is 16 Gigabytes. Solution:

1. Make sure the inner (build) table is the smaller of the two tables, say Dept.
2. Break the inner table into M buckets using a hash function. The ideal function ensures each bucket is small enough to fit in memory. Write these buckets to disk.
3. Break the outer table (Emp) into M buckets using the same hash function used in Step 2. Write these buckets to disk.
4. For i = 1 to M:
    a. Stage Bucket i of Dept in an in-memory hash table
    b. Read Bucket i of Emp and probe the hash table for joining records

Hash-Join: Not Enough Memory

- Divide-and-conquer: Records in bucket i of the inner table (Dept) may join with the records in bucket i of the outer table (Emp) only.
  - Use of a hash function has reduced 1 join into M small joins.

- The first bucket of the inner table (Dept) does not have to be written to disk, it can be staged in memory. Similarly, the first bucket of the outer table (Emp) is not required to be written to disk. It may probe the hash table of the inner table (Dept) for joining records.

Hash-Join: Multiple Nodes

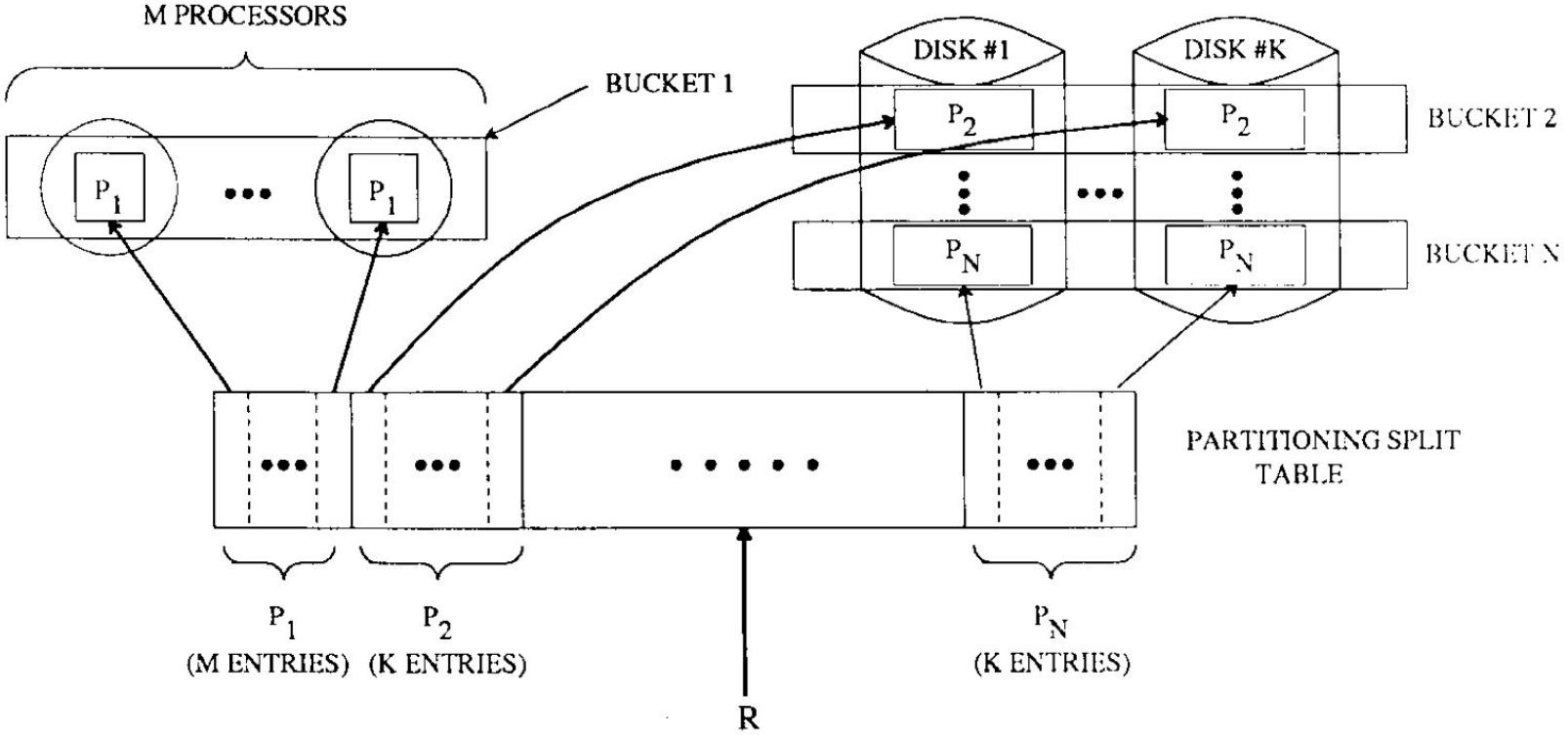


Fig. 8. Partitioning of R into N logical buckets for hybrid hash–join.

Software Modularity: Split Table

- Once a process begins execution, it continuously reads records from its input stream, operates on each tuple, and uses a split table to route the resulting tuple to the process indicated in the split table.
- When the process detects the end of its input stream, it first closes the output streams and then sends a control message to its scheduler process indicating that it has completed execution. Closing the output stream generates an "end of stream" message to each of the destination processes.

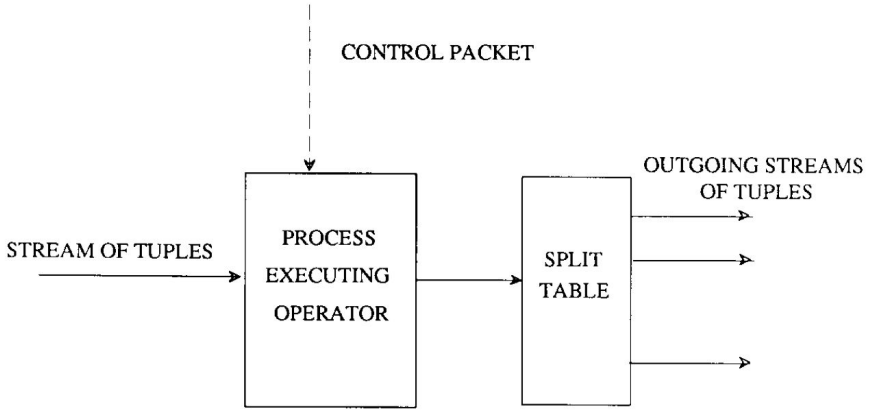


Fig. 3.

Value	Destination Process
0	(Processor #3, Port #5)
1	(Processor #2, Port #13)
2	(Processor #7, Port #6)
3	(Processor #9, Port #15)

Fig. 4. An example split table.

Node Failures: Interleaved Declustering

Maintain data availability by constructing a primary and a backup replica of fragments. How? Interleaved declustering.

	Cluster 0				Cluster 1			
Node	**0**	**1**	**2**	**3**	**4**	**5**	**6**	**7**
Primary Copy	R0	R1	R2	R3	R4	R5	R6	R7
Backup Copy		r0.0	r0.1	r0.2		r4.0	r4.1	r4.2
	r1.2		r1.0	r1.1	r5.2		r5.0	r5.1
	r2.1	r2.2		r2.0	r6.1	r6.2		r6.0
	r3.0	r3.1	r3.2		r7.0	r7.1	r7.2	

Fig. 10. Interleaved declustering (cluster size = 4).

Failure of Node 1 distributes its load on Nodes 0, 2 and 3. Why not increase cluster size to 8?

Node Failures: Chained Declustering

Maintain primary and backup replicas of a fragment on adjacent nodes.

Node	0	1	2	3	4	5	6	7
Primary Copy	R0	R1	R2	R3	R4	R5	R6	R7
Backup Copy	r7	r0	r1	r2	r3	r4	r5	r6

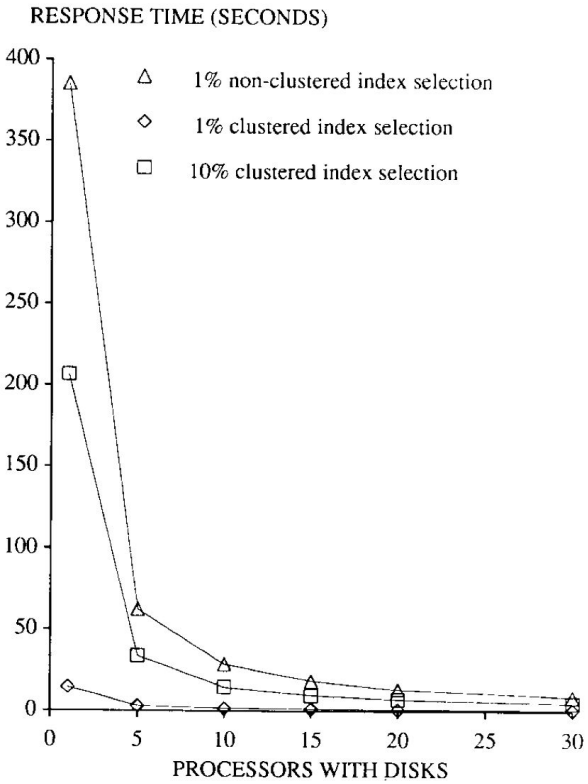
Node	0	1	2	3	4	5	6	7
Primary Copy	R0	---	$\frac{1}{7}$R2	$\frac{2}{7}$R3	$\frac{3}{7}$R4	$\frac{4}{7}$R5	$\frac{5}{7}$R6	$\frac{6}{7}$R7
Backup Copy	$\frac{1}{7}$r7	---	r1	$\frac{6}{7}$r2	$\frac{5}{7}$r3	$\frac{4}{7}$r4	$\frac{3}{7}$r5	$\frac{2}{7}$r6

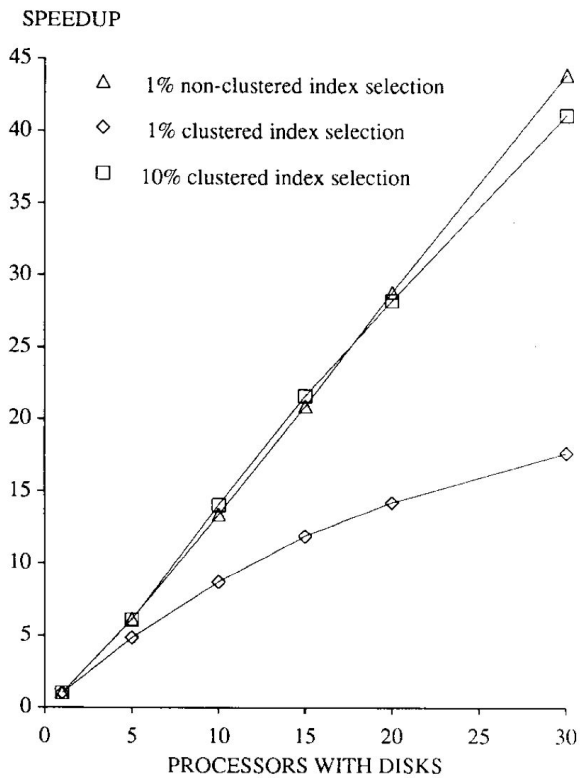
In the presence of failures, push a fraction of the load of a failed node to its adjacent node, balancing the workload across all nodes.

Select From one Table

- How is speedup computed?
- Why is speedup superlinear?

- TMP2 $\leftarrow \sigma_{attr>x}(R)$

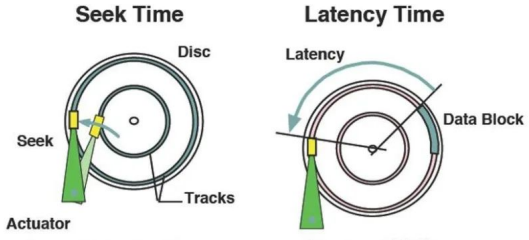




Superlinear Speedup: Multi-Zone Disk Drive

- Zones on the outer-regions of a disk drive provide a significantly higher bandwidth than the inner-regions of the disk drive. Transfer rate of each disk is higher with N disks when compared with 1 disk.
- Seek time is a function of the distance travelled by the disk head. The seek time is higher with one disk when compared with N disks.

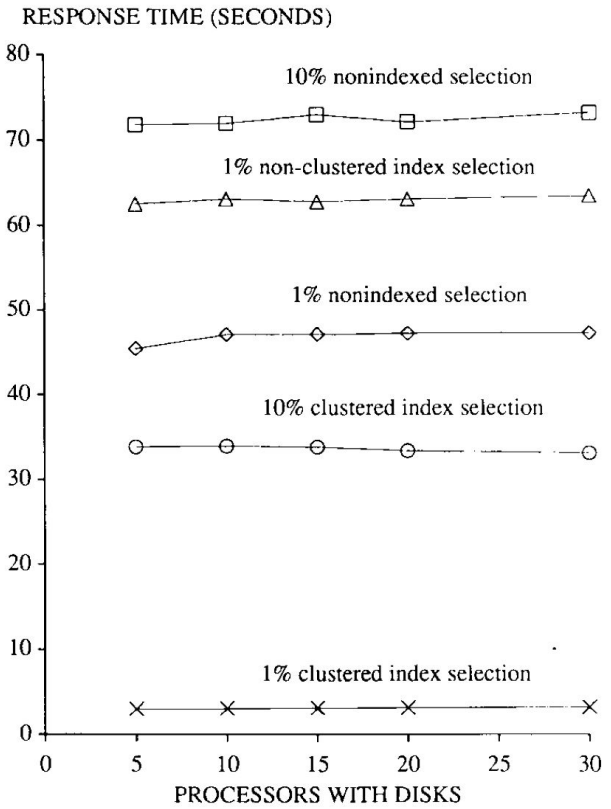
Zone #	Size (MB)	Transfer Rate (MBps)
0	139.7	4.17
1	67.0	4.08
2	45.7	4.02
3	45.3	3.97
4	43.9	3.88
5	42.6	3.83
6	41.3	3.74
7	56.4	3.60
8	39.2	3.60
9	37.5	3.50
10	51.5	3.36
11	35.0	3.31
12	33.8	3.22
13	46.5	3.13
14	31.8	3.07
15	30.7	2.99
16	41.8	2.85
17	28.0	2.76
18	27.6	2.76
19	37.1	2.62
20	25.0	2.53
21	33.5	2.43
22	25.4	2.33



Select Scaleup

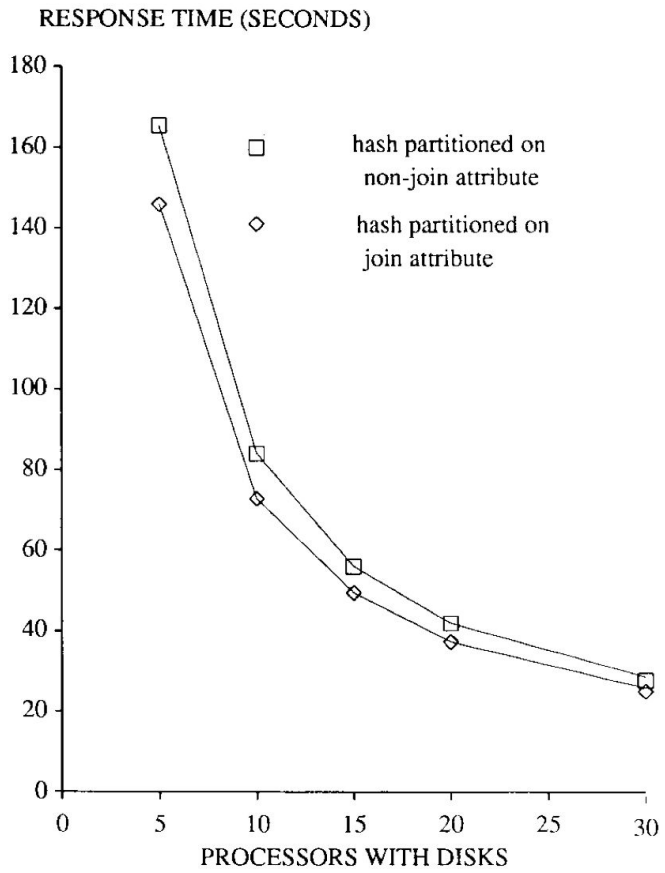
Size of input table increases from 1 Million records to 6 Million records.

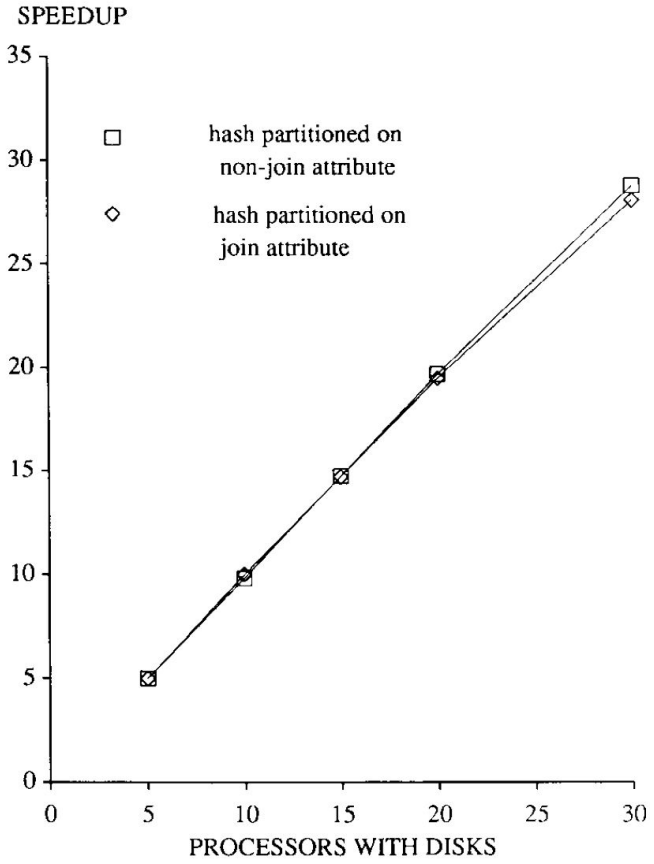
- TMP2 $\leftarrow \sigma_{attr>x}(R)$



Join: Speedup

- TMP2 $\leftarrow A \bowtie B'$





Gamma Contributions

- Alternative declustering strategies including Hybrid Range Partitioning and MAGIC (Next lecture).
- Hybrid hash-join algorithm.
- High availability of data using chained Declustering.
- A data flow execution paradigm.
- A modular and configurable software system using the split operator.
- Use of nodes with disk and nodes without disk.
- Experimental methodology for multi-node DBMSs: Speedup and Scaleup.
- SQL to hide parallelism.